C# 数据结构与算法
完整学习指导

从不变量与复杂度，到工程实现与经典题型
结合项目源码、测试和现代 C# 语法循序学习

.NET 11	C# 15	LeetCode 100 题

DataStructureAndAlgorithm 学习项目

生成日期：2026-11-14

编排说明

本 PDF 将当前生成入口选择的 7 份学习文档合并为一册。各篇按从基础方法、进阶结构和 API 契约，到经典题、高级专题与综合应用的顺序编排：先形成可复用的思考框架，再通过题目和真实场景训练算法选型与组合不变量。

> 源码仍是最终实现依据。阅读时建议同时打开对应的 C# 文件和 xUnit 测试，先手算样例，再运行测试，最后尝试在不看答案的情况下重写核心步骤。

目录

第一篇 基础学习路线与 C# 最佳实践 6

1. 这份指导解决什么问题 6
2. 项目知识地图 6
3. 环境与常用命令 8
4. 学习每个算法的固定步骤 9
5. 推荐学习顺序 10
6. C# 最佳实践指南 12
7. 测试最佳实践 15
8. 建议的练习方式 16
9. P0-P4、综合应用完成度与后续路线 17
10. 阶段验收清单 18

第二篇 进阶数据结构与算法 20

1. 使用方式 20
2. 进阶知识地图 20
3. 哈希表：从哈希码到平均 O(1) 21
4. 并查集：动态连通性 22
5. Trie：以前缀作为路径 22
6. LRU：组合数据结构 23
7. Fenwick 树与线段树 23

8. 图算法选择 24
9. 进阶动态规划 24
10. 字符串算法进阶 25
11. 数组、贪心和非比较排序 25
12. 组合搜索 26
13. 更深入的 C# 15 实践 26
14. 测试进阶方法 27
15. 第 1–8 周进阶学习计划 28
16. 第 1–8 周核心阶段验收 29
17. 四个正确性修复：为什么原实现会出错 29
18. 双向链表与数组双端队列 31
19. 开放寻址哈希表：Empty、Occupied、Deleted 三态 31
20. 桶排序与基数排序 32
21. 左倾红黑树：用局部规则维护全局高度界 32
22. 跳表：用概率换取简单的平衡逻辑 33
23. CI 与测试为什么这样组织 33
24. 第 9–12 周学习安排 34
25. 可选算法追踪与 Demo 为什么这样设计 34
26. 从示例测试到性质、状态机和变异测试 35
27. 索引优先队列：为什么需要键到下标的反向索引 35
28. 六个图算法应该怎样选择 36
29. 有序集合统一契约与比较器最佳实践 37
30. Aho-Corasick：失败指针为什么能一次扫描多个模式 37
31. 后缀数组与 LCP：为什么采用排名倍增 37
32. Sparse Table 与懒标记线段树的选择边界 38
33. Span 与 ArrayPool：优化必须连同所有权一起学习 38

34. 第 13–18 周学习顺序 . . . 38

第三篇 API 设计与比较器约定 . . . 40

1. 参数验证 . . . 40
2. 区间统一为半开形式 . . . 40
3. 相等与排序 . . . 40
4. 重复值与修改返回值 . . . 41
5. 输入与输出所有权 . . . 41
6. 数值契约 . . . 41
7. 可选诊断不污染核心算法 . . . 41
8. 测试与性能证据 . . . 41
9. C# 15 写法边界 . . . 42

第四篇 LeetCode 100 题题型地图 . . . 43

1. 学习目标 . . . 43
2. 完整题号清单 . . . 43
3. 数组与矩阵：20 题为什么这样做 . . . 44
4. 字符串：15 题为什么这样做 . . . 45
5. 链表与设计：14 题为什么这样做 . . . 46
6. 二叉树：15 题为什么这样做 . . . 46
7. 图与回溯：10 题为什么这样做 . . . 47
8. 动态规划、贪心和单调栈：7 题为什么这样做 . . . 48
9. C# 15 实现原则 . . . 48
10. 测试策略 . . . 49
11. 16 周学习顺序 . . . 50
12. 验收标准 . . . 50
13. P3 追加 14 题 . . . 50

第五篇 核心 19 题精讲 . . . 52

1. 目标与使用方式 . . . 52
2. 题目地图 . . . 52
3. 哈希表题型 . . . 53
4. 排序与双指针 . . . 53
5. 滑动窗口与字符串 . . . 54
6. 链表题型 . . . 55
7. 二叉树题型 . . . 56
8. 图与网格 . . . 56
9. 动态规划 . . . 57
10. 如何独立解题 . . . 57
11. 测试策略 . . . 58
12. 四周练习计划 . . . 58
13. 验收标准 . . . 59

第六篇 P3 数论、位运算与高级算法 . . . 60

1. P3 为什么不继续机械增加题量 . . . 60
2. P3 知识地图 . . . 60
3. 数论：为什么欧几里得算法有效 . . . 60
4. 位运算：什么时候位技巧是真正的状态表达 . . . 62
5. Johnson：稀疏图为什么不总用 Floyd . . . 62
6. Hopcroft-Karp：为什么一次寻找多条增广路 . . . 62
7. 欧拉路径：为什么答案要逆序构造 . . . 63
8. 倍增 LCA：为什么预处理祖先的 2 的幂 . . . 63
9. 最大流与最小割为什么要一起学习 . . . 63
10. Manacher：为什么镜像只能提供下界 . . . 63
11. 后缀自动机：状态为什么能代表多个子串 . . . 63
12. Rune：为什么 char 不等于一个用户可见字符 . . . 64

13. 三类高级动态规划为什么要分开学习 64

14. 新增 14 道不重复专题题 65

15. 结构化 Trace 和 Mermaid 为什么分层 65

16. CI 质量门禁为什么这样设置 66

17. 推荐八周学习顺序 66

18. P3 验收清单 66

第七篇 数据结构与算法综合项目实战 68

1. 为什么需要综合项目 68

2. 四个项目共用的分层原则 68

3. 项目一：城市即时配送 V2 69

4. 项目二：迷你搜索引擎 V2 70

5. 项目三：项目调度 V2 72

6. 项目四：迷你存储引擎 74

7. 四个项目共同展示的 C# 最佳实践 76

8. 把正确性升级为“持续可证明” 77

9. 推荐的八周学习路线 79

10. 可继续扩展的方向 80

11. 阶段验收清单 81

第一篇 基础学习路线与 C# 最佳实践

> 从复杂度、不变量和测试开始，建立线性结构、树、图、排序、动态规划与回溯的完整基础。

title: C# 数据结构与算法学习指导 description: 从复杂度、不变量和测试出发，按八个阶段系统学习本仓库的数据结构、算法与综合项目。 search: false

1. 这份指导解决什么问题

本仓库是一个 .NET 11 / C# 15 教学项目。目标不只是“记住算法代码”，而是建立以下能力：

1. 能说清数据结构维护了什么不变量。
2. 能从输入规模推导时间、空间复杂度。
3. 能独立处理空输入、重复元素、越界、溢出和无解情况。
4. 能用泛型、比较器、可空引用类型和单元测试写出符合 C# 习惯的实现。
5. 知道教学实现与生产代码之间的边界。

项目以类库为核心，并提供职责单一的算法 Demo、P3 追踪 Demo、基准项目和四个综合场景项目。Demo 用 JSON/Mermaid 展示状态，Benchmarks 用独立进程测量时间和分配；综合项目把多个已学结构组合成端到端应用，并通过统一、可选的追踪 sink 解释流水线。学习时先读源码与测试，再用 Demo 观察单步状态，最后进入配送、搜索、调度和存储场景解释“为什么选这些结构、它们如何共同维护不变量”；不要把输出、计时或 UI 逻辑塞回算法类。四个场景的目标函数、复杂度、限制与正确性证据见 综合项目实战学习指导。

完成本篇主干内容后，继续阅读 C# 数据结构与算法进阶学习指导，学习哈希表、并查集、Trie、LRU、区间结构和进阶算法。

需要按经典题型练习时，阅读 LeetCode 100 题完整学习指导。

新增模块应遵循 API 设计与比较器约定，其中集中说明空值、半开区间、比较器、集合重复、输入所有权和异常类型为什么这样统一。

2. 项目知识地图

主题	主要源码	应掌握的核心概念
单向链表	Linear/SinglyLinkedList.cs	节点引用、头尾指针、O(1) 头部操作
双向链表	Linear/DoublyLinkedList.cs	前驱/后继、O(1) 两端删除、双向链接不变量
栈	Linear/ArrayStack.cs	LIFO、动态数组、摊还复杂度
队列	Linear/CircularQueue.cs	FIFO、环形索引、扩容时保持逻辑顺序
双端队列	Linear/ArrayDeque.cs	两端 O(1) 操作、循环数组、逻辑/物理下标
堆	Heap/BinaryMinHeap.cs	完全二叉树、上浮、下沉、优先队列
查找	Searching/SearchAlgorithms.cs	二分查找、半开区间、Lower/Upper Bound
排序	Sorting/SortAlgorithms.cs、SequenceExtensions.cs	稳定性、原地排序、分治、C# 14 扩展块
字符串匹配	PatternMatching/StringPatternMatching.cs	朴素匹配、KMP、前缀函数
普通二叉树	Tree/NormalBinaryTree.cs	深度优先、广度优先、递归与迭代
搜索树	Tree/BinarySearchTree.cs	有序不变量、查找、插入、删除

主题	主要源码	应掌握的核心概念
AVL 树	Tree/AvlTree.cs	平衡因子、旋转、O(log n) 树高
红黑树	Tree/RedBlackTree.cs	左倾红链接、颜色翻转、黑高不变量
跳表	Tree/SkipList.cs	随机层高、多级索引、期望 O(log n)
B 树	Tree/BTree.cs	多路平衡、分裂、借位、合并
B+ 树	Tree/BPlusTree.cs	内部节点分隔键、叶节点有序记录、叶链范围扫描
线索二叉树	Tree/CluesBinaryTree.cs	利用空指针保存遍历前驱/后继
哈夫曼树	Tree/HuffmanTree.cs	贪心选择、前缀编码、最小堆
无权图	Graph/GraphWithAdjacencyList.cs 等	邻接表/矩阵、BFS、DFS
带权图	Graph/WeightedGraph.cs	权重、稀疏图、顶点比较规则
图算法	Graph/WeightedGraphAlgorithms.cs	Dijkstra、拓扑排序、环检测
动态规划	DynamicProgramming/DynamicProgrammingAlgorithms.cs	状态、转移、初始化、答案恢复
回溯	Backtracking/BacktrackingAlgorithms.cs	选择、约束、递归、撤销选择
哈希表	Hashing/SeparateChainingHashTable.cs、OpenAddressingHashTable.cs	链地址、线性探测、墓碑、负载因子、重新散列
概率结构	Probabilistic/StringBloomFilter.cs	位图、多哈希、允许假阳性但禁止假阴性
并查集	Set/DisjointSet.cs	路径压缩、按大小合并、动态连通性
Trie	Tree/Trie.cs	前缀共享、单词结束标记、安全删除
编辑距离索引	Tree/BkTree.cs	度量空间、三角不等式、近似词剪枝
缓存	Caching/LruCache.cs、LfuCache.cs	最近使用与最少使用淘汰、缓存一致性
区间结构	Range/FenwickTree.cs、SegmentTree.cs、SparseTable.cs、LazyRangeSumSegmentTree.cs、LazyRangeAddMaxSegmentTree.cs	lowbit、静态 O(1) 查询、单点/区间更新、懒标记
进阶图算法	Graph/AdvancedGraphAlgorithms.cs、ComprehensiveGraphAlgorithms.cs、MinCostFlowAlgorithms.cs	Bellman-Ford、生成树、全源最短路、A*、最大流、最小费用流
进阶动态规划	DynamicProgramming/AdvancedDynamicProgrammingAlgorithms.cs	编辑距离、LIS、空间压缩、结果恢复
进阶字符串	PatternMatching/AdvancedStringAlgorithms.cs、AhoCorasickMatcher.cs、SuffixArrayAlgorithms.cs	滚动哈希、Z 区间、多模式匹配、后缀索引与 LCP
内存与性能	Memory/SpanAlgorithms.cs、DataStructureAndAlgorithm.Benchmarks/	Span 零拷贝视图、ArrayPool 所有权、基准与分配诊断
数组模式	ArrayAlgorithms/ArrayAlgorithms.cs	哈希、单调队列、Kadane
贪心	Greedy/GreedyAlgorithms.cs	区间调度、交换论证
非比较排序	Sorting/NonComparisonSortAlgorithms.cs	计数、桶、LSD 基数排序与输入域约束
组合搜索	Backtracking/CombinatorialAlgorithms.cs	括号剪枝、幂集生成
LeetCode 100 题	LeetCode/Classic*Problems.cs、Additional*Problems.cs	题型识别、不变量、逐题测试、API 适配
综合项目	DataStructureAndAlgorithm.Scenarios.CityDelivery/、MiniSearch/、ProjectScheduling/、MiniStorage/	组合最小费用流、位置索引/BM25、精确调度、B+ 树/WAL，并区分事实来源与加速层
统一追踪	Diagnostics/AlgorithmTracing.cs、Diagnostics/MermaidTraceRenderer.cs	可选 sink、schema version、只读状态快照、稳定步骤、JSON/Mermaid 与领域结果解耦

测试项目保持相同主题目录。遇到看不懂的实现时，先看测试构造了什么输入、期待什么输出，再回到源码追踪状态变化。

3. 环境与常用命令

仓库不锁定 SDK 版本，装有 .NET 11 或更高版本的 SDK 即可构建，使用的是本机已装的最新 SDK；类库、测试、演示、基准和综合场景项目统一以 .NET 11 / C# 15 为基线。确认环境：

```powershell
dotnet --info
```

运行全部测试：

```powershell
dotnet test DataStructureAndAlgorithm.slnx --configuration Release
```

观察排序、Dijkstra、红黑树和 Huffman 的 JSON 步骤：

```powershell
dotnet run --project DataStructureAndAlgorithm.Demo -- all
```

运行四个综合项目，观察同一种结构在不同领域约束下承担的职责。无参数模式保留领域结果 JSON；另外两种模式共享同一个算法实现，只改变观察方式：

```powershell
dotnet run --project DataStructureAndAlgorithm.Scenarios.CityDelivery
dotnet run --project DataStructureAndAlgorithm.Scenarios.MiniSearch
dotnet run --project DataStructureAndAlgorithm.Scenarios.ProjectScheduling
dotnet run --project DataStructureAndAlgorithm.Scenarios.MiniStorage

dotnet run --project DataStructureAndAlgorithm.Scenarios.CityDelivery -- --trace json
dotnet run --project DataStructureAndAlgorithm.Scenarios.MiniSearch -- --trace mermaid
dotnet run --project DataStructureAndAlgorithm.Scenarios.ProjectScheduling -- --trace json
dotnet run --project DataStructureAndAlgorithm.Scenarios.MiniStorage -- --trace json
```

--trace json 的顶层固定为 { result, trace }，每个事件带 schema version，适合程序处理和契约测试；--trace mermaid 适合把有序事件直接放入学习笔记。追踪默认关闭，是为了让观察功能不改变领域 API，也不强迫正常路径承担状态字典与快照开销；启用时使用 best-effort 装饰器隔离普通接收器异常，避免日志失败反过来破坏领域提交语义。

不要一开始就背综合项目的完整实现。先写下输入契约和跨结构不变量，再逐段替换为自己实现的数据结构；这样能分辨“算法本身正确”和“组合边界正确”是两层不同证据。

先用短任务确认基准可启动，再去掉 --job Dry 做正式测量：

```powershell
dotnet run --project DataStructureAndAlgorithm.Benchmarks -c Release -- --filter
"*Sorting*" --job Dry
```

只运行一个主题：

```powershell
dotnet test DataStructureAndAlgorithm.slnx --filter FullyQualifiedName~SortingTest
```

仓库的 .github/workflows/ci.yml 会在 push 和 pull request 上使用相同的 .NET 11 Release 流程执行 restore、build、test 与覆盖率收集。把构建和测试拆开并在后续命令使用 --no-restore、--no-build，可以明确区分“依赖恢复失败” “编译失败”和“行为测试失败”，也避免重复工作。

学习时建议先使用 Debug 断点或 Demo 追踪理解过程，完成后再跑 Release 全量测试。算法性能比较不能只看一次运行时间；仓库已经把 BenchmarkDotNet 放在独立项目中，避免在单元测试里用 Stopwatch 得出受 JIT、预热、GC 和机器负载影响的结论。

4. 学习每个算法的固定步骤

4.1 明确输入、输出和契约

先回答：

- 输入允许为空吗？
- 是否允许重复元素？
- 是否原地修改输入？
- 比较顺序来自哪里？
- 找不到答案时返回 -1、null、空集合，还是抛异常？
- 输入不满足前置条件时由谁负责？

例如二分查找的前置条件是序列已按同一个 IComparer<T> 排序。每次查找都先验证有序性需要 O(n)，会失去二分查找的复杂度优势，因此该条件应写入文档，由调用方保证。

4.2 找到不变量

不变量是算法执行过程中始终为真的条件：

- 栈：有效元素位于 [0, Count)，栈顶是 Count - 1。
- 循环队列：第 i 个逻辑元素位于 (_head + i) % Capacity。
- 最小堆：任意父节点都不大于其子节点。
- BST：左子树元素小于节点，右子树元素大于节点。
- AVL：除 BST 不变量外，每个节点左右子树高度差不超过 1。
- 红黑树：根为黑色、红节点没有红孩子、右侧没有红链接、任意根到空叶路径黑节点数相同。
- 开放寻址哈希表：查找必须越过 Deleted 墓碑，只能在真正 Empty 槽位终止。
- 双向链表：head.Previous 和 tail.Next 为空，正向与反向链接互相对应。
- Dijkstra：从优先队列取出的当前最小有效距离不会再被非负边缩短。
- 动态规划表：填 dp[i, j] 时，它依赖的子问题已经计算完成。

调试时不要只盯最终输出；在循环或递归的关键位置检查不变量，错误会更容易定位。

4.3 手算最小例子

至少覆盖：

1. 空集合或最小合法输入。
2. 单个元素。
3. 重复元素。
4. 已有序与逆序输入。
5. 触发关键分支的输入，例如 AVL 四种旋转、循环队列绕回、B 树合并。

6 非法输入和无解输入。

4.4 推导复杂度

分别写时间和额外空间复杂度，并说明平均、最坏或摊还情形。

常见判断：

- 连续嵌套两层、每层最多 n 次，通常是 $O(n^2)$。
- 每次把问题规模减半，通常出现 O(log n)。
- 分成两个一半子问题并线性合并，通常是 O(n log n)。
- 图遍历访问每个顶点和每条边一次，是 O(V + E)。
- 递归函数除显式容器外，还要计算调用栈空间。

复杂度忽略常数但不等于性能。缓存局部性、分支预测、对象分配和比较器成本都会影响实际运行速度。

4.5 用测试固定理解

一个好的算法测试应表达一个规则，而不是复制实现过程。比如测试最小堆时，连续出队结果必须有序，比断言某一时刻内部数组恰好长什么样更稳健。

5. 推荐学习顺序

第一阶段：语言与复杂度基础

先掌握数组、引用、值类型、泛型、委托、异常、IEnumerable<T>、Big-O。练习手写数组遍历、交换、最大值和前缀和。

验收标准：能解释数组随机访问为什么是 O(1)，链表按下标访问为什么是 O(n)。

第二阶段：线性数据结构

依次阅读：

1 SinglyLinkedList<T>

2 DoublyLinkedList<T>

3 ArrayStack<T>

4 CircularQueue<T>

5 ArrayDeque<T>

重点观察头尾指针、Count、空结构和扩容。独立练习：

- 链表反转。
- 快慢指针检测环。
- 比较单向链表和双向链表在尾部删除时的差异。
- 用两个栈实现队列。
- 用队列实现二叉树层序遍历。
- 手算双端队列在物理数组末尾绕回到下标 0 的过程。

第三阶段：查找与排序

先学插入、选择、冒泡，理解循环不变量；再学归并、快速和堆排序，理解分治与堆。

需要能够比较：

算法	最好	平均/最坏	额外空间	稳定
冒泡	O(n)	$O(n^2)$	O(1)	是
选择	$O(n^2)$	$O(n^2)$	O(1)	否
插入	O(n)	$O(n^2)$	O(1)	是
归并	O(n log n)	O(n log n)	O(n)	是
快速	O(n log n)	最坏 $O(n^2)$	平均 O(log n) 栈	否
堆排序	O(n log n)	O(n log n)	O(1)	否

随后学习 BinarySearch、LowerBound、UpperBound。重点掌握 [left, right) 半开区间，避免死循环和边界越界。

最后学习非比较排序：计数排序依赖较窄整数值域；桶排序依赖较均匀分布并在桶内使用比较排序；LSD 基数排序按低位到高位进行稳定分配。本项目的基数排序用 value ^ int.MinValue 翻转符号位，从而覆盖完整的有符号 int 值域。

第四阶段：树与堆

推荐顺序：普通二叉树 → BST → AVL → 红黑树 → 最小堆 → 哈夫曼树 → B 树 → 线索二叉树 → 跳表。

不要把“二叉堆”和“二叉搜索树”混为一谈：堆只保证父子顺序，不能高效查找任意值；BST 保证整棵左、右子树的有序关系。

练习时为结构写验证函数：

- BST 中序遍历严格递增。
- AVL 每个节点平衡因子合法，保存高度与实际高度一致。
- 红黑树根黑、无连续红、红链接左倾且所有路径黑高一致。
- B 树节点键数、孩子数、键区间和叶子深度符合约束。
- 跳表第 0 层严格有序，高层节点一定也出现在全部更低层。

第五阶段：字符串与图

先用朴素匹配理解问题，再学习 KMP 如何利用已经匹配的信息减少回退。

图按以下顺序学习：

1. 邻接矩阵与邻接表。
2. BFS 与 DFS。
3. 拓扑排序及有向环检测。
4. Dijkstra 非负权最短路。
5. 继续扩展：Prim、Floyd-Warshall、强连通分量、桥与割点、A*。

Dijkstra 遇到负权边会抛异常。负权边不是“小概率特殊情况”，而是算法前置条件不成立，应改用 Bellman-Ford 等算法。

第六阶段：动态规划与回溯

动态规划不要从背代码开始。对每个问题写出：

1. 状态含义。
2. 状态转移方程。
3. 初始状态。
4. 填表顺序。
5. 最终答案位置。
6. 是否需要恢复具体选择。

本项目中的斐波那契、最长公共子序列、0/1 背包和零钱兑换分别展示了空间压缩、二维状态、答案恢复和不可达哨兵。

回溯以 n 皇后为例。每层递归选择一行中的列，集合用于剪枝，返回上一层前必须撤销所有状态修改。

第七阶段：四个综合项目

按“配送 V2 → 搜索 V2 → 调度 V2 → 迷你存储”的顺序学习。前三个项目分别训练网络优化、信息检索和资源约束；存储项目再把有序索引、概率预筛、缓存和恢复协议组合到同一读写路径。

每个项目不要从 Program.cs 的最终 JSON 开始背诵，而要依次回答：

1. 业务目标是什么，多个目标的优先级是什么？
2. 哪个结构保存事实，哪个结构只是加速层？
3. 跨模块不变量是什么，哪个简单模型可以充当 oracle？
4. 复杂度瓶颈在哪里，教学实现距离生产系统还缺什么？

完整八周路线和手算练习见 综合项目实战学习指导。

第八阶段：持续正确性与有证据的优化

把回归、不变量、模型/差分、FsCheck、状态机、trace contract、跨平台 CI、变异测试和 BenchmarkDotNet 串成持续证据链。先证明优化前后的领域结果等价，再比较时间和分配趋势；不要用覆盖率代替正确性，也不要用共享 CI 上的一次毫秒数字决定优化成败。

验收标准：能够为一个新模块同时写出契约、不变量、独立 oracle、异常测试、跨平台输出约定和基准假设，并解释每种证据能发现什么、不能证明什么。

6. C# 最佳实践指南

6.1 开启可空引用类型

项目使用 <Nullable>enable</Nullable>。T?、空检查和编译器警告是在表达 API 契约，不应通过到处添加 ! 来消除警告。

适合使用 ! 的情形很少，例如数组槽位已在逻辑上移除，需要写入 default! 以释放引用，而调用方永远不会访问该槽位。此时应有注释说明不变量。

6.2 参数验证应靠近公共 API 边界

现代 .NET 可使用：

CSHARP

```
ArgumentNullException.ThrowIfNull(values);
ArgumentOutOfRangeException.ThrowIfNegative(capacity);
```

参数本身非法用 ArgumentException 系列；对象当前状态不允许操作用 InvalidOperationException；查找普通失败通常返回约定值，不要把异常当正常分支。

6.3 泛型算法使用比较器

不要假设元素一定是 int，也不要强制所有类型实现 IComparable<T>。推荐接受可选的 IComparer<T>：

CSHARP

```
comparer ??= Comparer<T>.Default;
```

判断“身份相等”使用 IEqualityComparer<T>；判断“排序先后”使用 IComparer<T>。两者语义不同。图一旦接受自定义相等比较器，内部所有字典、去重和路径恢复都必须沿用同一个比较器。

6.4 暴露最小能力接口

只读输入优先考虑 IReadOnlyList<T>；需要原地赋值的排序算法使用 IList<T>；只需枚举一次则用 IEnumerable<T>。参数接口越小，调用方选择越多，方法契约也越清楚。

不要直接暴露可变的内部 List<T>。可以返回 IReadOnlyList<T>，但要注意它只是接口只读，不一定是数据快照。安全边界要求更高时应返回副本或不可变集合。

6.5 封装并维护不变量

节点、数组和索引通常是实现细节：

- 节点类型可以是私有嵌套类。
- Count 使用 private set。
- 不让调用方直接修改堆数组或链表的 Next。
- 公共方法完成后，结构必须立即回到合法状态。

教学项目可以公开少量状态帮助观察，但要明确这是否会允许调用方破坏结构。

6.6 优先清楚的代码，再做有证据的优化

先实现容易证明正确的版本，再用基准数据判断是否需要：

- Span<T> 或 Memory<T>。
- ArrayPool<T> 减少大数组分配。
- 结构体避免对象分配。
- 迭代替代深递归。
- 专用比较器减少委托开销。

这些工具会增加生命周期、所有权或复制语义的复杂度。没有测量结果时，不要为了“看起来高性能”牺牲正确性和可读性。

6.7 理解值类型与引用类型

record struct KnapsackItem 适合小型、不可变、值语义的数据。树节点包含可变引用和身份，适合 class。大型可变结构体容易产生隐式复制，不适合作为节点。

record 自动提供值相等和易读的 ToString()，适合算法结果 DTO；它不会自动让其中引用的集合变成不可变集合。

6.8 使用现代语法但保持意图清晰

项目显式使用 C# 15，并在适合的位置采用现代语法：

- 集合表达式 [] 简化数组、列表和空集合初始化。
- 主构造函数适合依赖少、初始化关系清晰的节点类型。
- record / record struct 表达值语义的数据和算法结果。
- 模式匹配、元组交换、局部函数和 using var 减少样板代码。
- C# 14 扩展块把同一接收者类型的扩展成员组织在一起。

Sorting/SequenceExtensions.cs 是扩展块的实际示例：

CSHARP

```
public static class SequenceExtensions
{
    extension<T>(IEnumerable<T> source)
    {
        public bool IsSorted(IComparer<T>? comparer = null) { /* ... */ }
    }
}
```

C# 14 的 field 关键字、空条件赋值等语法只应在确实减少字段或分支噪声时使用。算法项目不能为了展示语法而隐藏关键状态变化。

选择标准不是“语法最新”，而是它是否让契约和算法意图更清楚。团队项目还应由 LangVersion 和 CI 固定语言版本，避免不同 SDK 产生不一致结果。

算法中的关键状态最好使用有意义的名称，例如 remainingCapacity，不要为了短而全部写成 i、j。只有很小、含义明确的循环下标适合短名称。

6.9 注释解释原因和约束

低价值注释：

CSHARP

```
index++; // index 加一
```

高价值注释应解释：

- 为什么选择半开区间。
- 为什么相等时先取归并左侧元素才能保持稳定。
- 为什么清空数组槽位可以释放引用。
- 为什么优先队列中存在过期条目以及如何跳过。
- 某个前置条件不满足会导致什么错误。

公共 API 使用 XML 文档；复杂循环和不变量使用局部注释。注释必须随代码更新，错误注释比没有注释更危险。

6.10 避免隐藏的多次枚举

IEnumerable<T> 可能来自数据库、网络或一次性生成器。不要在不知道来源时反复调用 Count()、Any() 和遍历。若算法确实需要随机访问，可以在入口明确物化一次：

```csharp
var items = source as IReadOnlyList<T> ?? source.ToArray();
```

同时要意识到物化会消耗 O(n) 额外空间。

6.11 处理整数与浮点边界

中点使用 left + (right - left) / 2，避免 left + right 溢出。斐波那契明确限制到 long 可表示的 F(92)。金额、容量和数组长度通常应拒绝负数。

浮点权重需要拒绝 NaN 和无穷值。涉及货币时通常用 decimal 或最小货币单位的整数，不应直接套用 double。

6.12 优先使用标准库完成生产任务

手写集合和排序用于学习。生产代码通常优先：

- List<T>、LinkedList<T>、Stack<T>、Queue<T>。
- Dictionary<TKey,TValue>、HashSet<T>。
- PriorityQueue<TElement,TPriority>。
- Array.Sort、List<T>.Sort、LINQ 的 OrderBy。

标准库实现经过广泛测试和优化。只有明确的性能、内存布局、领域约束或教学需求才手写。

7. 测试最佳实践

7.1 一个测试表达一个行为

测试名称建议采用：Method_Condition_ExpectedBehavior。准备、执行、断言三个阶段保持清晰。

7.2 同时测试结果和不变量

排序不仅检查示例结果，还要覆盖空集合、单元素、重复数、负数和自定义比较器。树和堆除了返回值，还要验证结构约束。

7.3 不复制被测算法

测试快速排序时不要在测试里再写一遍快速排序。可以使用明确的期望数组，或用标准库排序产生参考结果，再对随机样本进行差分测试。

7.4 随机测试必须可复现

固定随机种子，并在失败信息中输出种子和输入。无种子的偶发失败很难调试。

7.5 单元测试不是性能测试

CI 机器负载不稳定。单元测试负责正确性，BenchmarkDotNet 负责预热、重复、统计和运行时诊断。

7.6 用性质测试验证规则族

示例测试回答“这个输入是否正确”，FsCheck 性质测试回答“这一族输入是否始终满足同一规则”。综合场景应覆盖：配送的最大流量与最小费用、搜索的位置列表与 BM25 排序、调度的依赖/容量与精确最优性、存储的操作序列与恢复等价。测试数量不应成为目标；每条性质都应能说清它覆盖的规则和独立 oracle。

性质测试应遵循四条规则：

1. 把随机值规范化为合法的小输入，让失败样本能够被 FsCheck 缩减到可读反例。
2. oracle 使用完整排序、直接扫描、简单数组、SortedDictionary 或小规模穷举等不同思路，不复制被测优化算法。
3. 先验证不变量，再验证方便展示的聚合结果，避免多个错误字段彼此“自证”。
4. 保留固定回归与异常测试；随机样本不是穷举证明，也不擅长精确验证异常类型和参数名。

7.7 追踪也需要测试

观察代码如果不受约束，也会产生误导证据。当前测试会确认：

- 注入 IAlgorithmTraceSink 前后领域结果相同；
- 步骤从 1 连续递增，事件停留在阶段和领域决策粒度；
- 收集器复制状态并只读公开，后续修改原字典不会篡改历史；
- Mermaid 按 Ordinal 排序 state key，并稳定转义引号、换行和反斜线；
- JSON 顶层保持 { result, trace }，事件 schema version 明确且可验证；
- 四个 CLI 的普通、JSON、Mermaid 和非法模式保持各自约定的退出码与输出通道；Mermaid 额外保证恰好一个平台原生末尾换行。

追踪测试只证明观察契约，不证明最短路、匹配、调度或恢复本身正确。算法正确性仍由示例、性质、差分和不变量测试共同承担。

7.8 建立持续运行的证据链

“持续可证明”不是把测试称为形式化证明，而是让每次修改都重新运行彼此独立的证据：

1. 回归测试固定已知边界和历史缺陷。
2. 结构不变量检查内部状态是否自洽。
3. 模型、差分和状态机测试把优化实现与简单参考模型逐步比较。
4. 跨平台 CI 在 Windows/Linux 暴露路径、换行、大小写和区域性差异。
5. 变异测试检查断言能否识别有意义的错误。
6. BenchmarkDotNet 保存时间和分配趋势，但不在共享 CI 上设置脆弱的毫秒硬阈值。

覆盖率只说明哪些代码被执行，变异分数只说明现有变体是否被测试识别，基准只说明性能；三者都不能替代领域契约和独立 oracle。完整证据矩阵见 综合项目实战学习指导。

8. 建议的练习方式

对每个主题完成三轮：

1. 阅读：逐行跟踪示例输入，写出每一步状态。
2. 默写：只看 API 和测试，独立实现。
3. 变式：修改约束，例如降序比较器、稳定排序、返回路径、恢复选择。

每个实现完成后回答：

- 不变量是什么？
- 最坏输入是什么？

- 复杂度是什么?
- 哪些地方可能溢出或越界?
- API 对空值和非法状态怎么处理?
- 能否用 BCL 类型完成，为什么这里仍然手写?

9. P0-P4、综合应用完成度与后续路线

仓库已经完成 P0/P1/P2：正确性修复、核心数据结构扩展、算法追踪、FsCheck 性质测试、状态机差分测试、BenchmarkDotNet、覆盖率与变异测试，以及高级图、字符串、区间和内存专题。

P3 继续补齐此前缺失的数学推理、算法族比较和可视化证据，并保持“不盲目堆数量”的原则。

P3 已完成内容

- 数论与位运算：gcd/lcm、扩展欧几里得、模快速幂、模逆元、筛法、质因数分解、矩阵快速幂、置位与子集枚举。
- 图：Johnson、Hopcroft-Karp、欧拉路径、倍增 LCA，以及可恢复最小割并输出步骤的 Dinic。
- 字符串：Manacher、后缀自动机和基于 Rune 的 Unicode 滑动窗口。
- 动态规划：矩阵链区间 DP、树上最大权独立集和 Held-Karp 状态压缩 TSP。
- 题库：新增 14 道与原 100 题不重复的数值、位运算和高级专题题。
- 可视化：P3 Demo 支持 JSON 与 Mermaid，展示余数、区间分割、层次图、增广流和最小割。
- 工程质量：统一格式、默认分析器、编译器警告、80% 行覆盖率和 70% 分支覆盖率门禁。

完整推导、源码索引、逐题解释和八周计划见 P3 数论、位运算与高级算法学习指导。

综合应用阶段已完成

- 城市配送 V2：先最大化派单数量，再以最小费用最大流最小化总旅行时间；最大匹配成为第一层目标的独立证据。
- 迷你搜索 V2：位置倒排表支持短语和邻近检索，BM25 改善相关性排序，BK-tree 用编辑距离三角不等式剪枝纠错候选。
- 项目调度 V2：懒标记树同时维护区间最大/最小值并用 Int128 安全组合延迟增量，小规模精确解为贪心工期提供最优性 oracle。
- 迷你存储引擎：B+ 树承担可惰性早停的有序事实索引，Bloom 与 LFU 分别加速负查询和热点读取，WAL/tombstone 及尾部提交标记提供可重放的恢复证据。

这一阶段的重点不是记住四套代码，而是练习从目标函数和领域约束选择结构：先定义跨模块不变量，再让每个结构只承担自己擅长的职责，最后用端到端证据同时验证结果、最优目标、依赖、容量和恢复等约束。详细场景、复杂度、限制和扩展练习集中在 综合项目实战学习指导。

P4：从可观察性升级为持续可证明

P4 不只增加结构，也把四个综合项目的正确性组织成可持续运行的证据链：

- 统一依赖 IAlgorithmTraceSink，默认 null；领域结果不包含诊断字段，开启观察前后使用同一算法路径，普通接收器异常由 BestEffortAlgorithmTraceSink 隔离。
- 四个控制台统一支持 --trace json 与 --trace mermaid；JSON 顶层固定为 { result, trace }，事件带显式 schema version。
- 事件只覆盖流水线阶段和领域决策；数值使用 InvariantCulture，Mermaid state key 按 Ordinal 排序，输出可跨平台复现。

- CollectingAlgorithmTraceSink 复制只读快照，避免可变字典篡改已经发生的历史事件。
- 模型、性质、差分和状态机测试使用简单数组、完整排序、小规模穷举或标准有序字典作为独立 oracle。
- CI 在 Windows/Linux 运行相同 Release 契约，BenchmarkDotNet 记录趋势和分配，不使用受机器噪声影响的硬时间阈值。

这样组织是因为结构越多，组合边界越容易隐藏错误。追踪回答“错误发生在哪个阶段”，性质和差分回答“问题是否存在于输入族”，跨平台 CI 防止环境契约漂移，基准趋势回答“优化是否仍值得”；它们互相补充而不彼此替代。

P4 后续路线

- 数据结构：Treap、可回滚并查集、持久化线段树、LSM Tree 与磁盘页模型。
- 算法：计算几何、二维区间结构、树链剖分，以及 cost-scaling、容量缩放或整数费用版本的最小费用流。
- 工程：显式栈 DFS、枚举器修改版本、WAL 校验/checkpoint、性能历史基线，并继续控制追踪事件粒度。
- 题库：只按新算法族加入配套题，不再以凑整数题量为目标。

“数据结构与算法”没有绝对完成状态。每增加一个主题，都应同时加入选择理由、核心不变量、复杂度推导、边界契约、正常/异常测试，以及适合该主题的差分、性质或变异证据。

10. 阶段验收清单

完成本仓库主干学习后，应能够：

- 从零实现链表、栈、队列、堆和二分查找。
- 实现双端队列，并解释循环数组扩容为什么必须按逻辑顺序复制。
- 比较六种排序的复杂度、稳定性和适用场景。
- 说明计数、桶和基数排序分别依赖什么输入条件。
- 写出前、中、后、层序遍历并说明递归栈空间。
- 解释 BST 退化原因与 AVL 旋转如何恢复平衡。
- 解释红黑树如何用颜色约束限制树高，并比较跳表的随机平衡策略。
- 比较链地址与开放寻址，说明墓碑删除为什么不可省略。
- 根据图的稠密度选择邻接表或邻接矩阵。
- 写出 BFS、DFS、拓扑排序和 Dijkstra，并说明各自前置条件。
- 能根据权重、查询次数和图结构选择 Bellman-Ford、Floyd、A*、Prim/Kruskal、Tarjan 或 Dinic，并说明为什么不能互换。
- 区分性质测试、状态机差分测试、变异测试、覆盖率和性能基准各自能证明什么、不能证明什么。
- 设计可缩减的小输入生成器，并用不同于被测实现的简单 oracle 验证综合场景不变量。
- 解释可选追踪 sink 为什么不应污染领域结果、best-effort 异常隔离为何避免“已提交却报失败”，以及默认关闭为什么能降低正常路径成本。
- 使用 --trace json 与 --trace mermaid 观察四个综合项目，并说明结构化数据与流程图的不同用途。
- 验证追踪前后领域结果一致，并解释 schema version、快照复制、InvariantCulture、Ordinal key 排序和特殊字符转义的必要性。
- 解释最小费用最大流为何能在最大派单数下继续优化总旅行时间。
- 从位置倒排表推导短语/邻近检索，并解释 BM25 和 BK-tree 的适用边界。

- 写出懒标记区间加/最大值不变量，并区分贪心可行解与小规模精确最优解。
- 说明 B+ 树、Bloom、LFU 和 WAL 在迷你存储引擎中分别承担事实索引、概率预筛、热点缓存和恢复证据。
- 解释 AC 自动机失败指针、后缀数组排名倍增、Sparse Table 幂等性和线段树懒标记为何正确。
- 正确管理 Span 与 ArrayPool 的内存所有权，并用基准数据决定是否采用池化版本。
- 为动态规划问题定义状态、写转移并恢复具体解。
- 用回溯模板表达选择、剪枝和撤销。
- 使用泛型比较器、可空引用类型、异常契约和 xUnit 测试组织 C# 算法代码。
- 知道何时应使用 .NET 标准集合，而不是把教学实现直接带入生产环境。

真正掌握的标志不是看懂代码，而是面对新约束时仍能从契约、不变量和复杂度重新推导出正确实现。

第二篇 进阶数据结构与算法

> 深入两种哈希策略、红黑树、跳表、非比较排序、区间结构与进阶算法，并补充 C# 15 工程实践。

title: C# 数据结构与算法进阶学习指导 description: 用十八周路线学习进阶数据结构、图与字符串算法、正确性证据、追踪和内存优化。 search: false

需要把进阶结构应用到经典题型时，继续阅读 LeetCode 100 题完整学习指导。

1. 使用方式

本篇建立在主学习指导的线性结构、排序、树、基础图、动态规划和回溯之上，重点训练三种能力：

1. 把基础结构组合成真实组件，例如“字典 + 双向链表 = LRU”。
2. 根据操作类型和输入约束选择区间结构、图算法或字符串算法。
3. 使用 C# 15 清楚表达泛型约束、比较规则、所有权、异常契约和测试。

每个专题建议按以下顺序学习：

1. 只看 API，写出输入、输出和复杂度预期。
2. 阅读源码注释，手算最小输入下的状态变化。
3. 阅读测试，区分正常、边界、异常和不变量断言。
4. 隐藏实现独立重写，再运行相同测试。
5. 完成章节练习，并解释为什么实现正确。

2. 进阶知识地图

主题	源码	核心不变量
链地址哈希表	Hashing/SeparateChainingHashTable.cs	条目位于当前容量计算出的桶中
开放寻址哈希表	Hashing/OpenAddressingHashTable.cs	探测链在 Empty 终止，但必须越过 Deleted 墓碑
双向链表、双端队列	Linear/DoublyLinkedList.cs、ArrayDeque.cs	双向链接一致；逻辑顺序不受数组绕回影响
并查集	Set/DisjointSet.cs	每个集合由唯一自指根代表
Trie	Tree/Trie.cs	根到节点的路径表示一个前缀
LRU	Caching/LruCache.cs	字典节点与双向链表节点一一对应
Fenwick 树	Range/FenwickTree.cs	节点保存 lowbit 决定的区间和
线段树	Range/SegmentTree.cs	父节点等于左右孩子的 combine 结果
Bellman-Ford	Graph/AdvancedGraphAlgorithms.cs	第 k 轮得到最多使用 k 条边的最短距离
Kruskal	Graph/AdvancedGraphAlgorithms.cs	已选边不成环，并查集描述连通分量
编辑距离、LIS	DynamicProgramming/AdvancedDynamicProgrammingAlgorithms.cs	DP 状态和前驱关系一致
Rabin-Karp、Z	PatternMatching/AdvancedStringAlgorithms.cs	滚动哈希窗口和最右匹配区间正确
数组模式	ArrayAlgorithms/ArrayAlgorithms.cs	候选队列单调，累计状态含义固定
区间调度	Greedy/GreedyAlgorithms.cs	已选区间兼容且结束时间尽可能早

主题	源码	核心不变量
计数排序	Sorting/NonComparisonSortAlgorithms.cs	counts 保存每个整数出现次数
桶排序、基数排序	Sorting/NonComparisonSortAlgorithms.cs	桶区间有序；每一位分配保持稳定
红黑树	Tree/RedBlackTree.cs	根黑、红链接左倾、无连续红、黑高一致
跳表	Tree/SkipList.cs	第 0 层全量有序，高层是低层的有序子集
组合搜索	Backtracking/CombinatorialAlgorithms.cs	递归返回前恢复调用前状态

3. 哈希表：从哈希码到平均 O(1)

3.1 映射过程

哈希表把任意键映射到数组桶：

```text
key -> comparer.GetHashCode(key) -> 非负整数 -> 对桶数量取模
```

哈希码不是唯一编号。不同键得到相同哈希码，或取模后落入同一个桶，都是正常的哈希冲突。最终必须使用 IEqualityComparer<TKey>.Equals 确认键是否相等。

3.2 链地址法与负载因子

链地址法让每个桶保存一组 Entry。查找只扫描目标桶。负载因子是：

```text
Count / Capacity
```

负载过高时，单桶平均条目增多。本项目在加入新元素会超过 0.75 前扩容。

3.3 为什么扩容必须重新散列

桶下标包含 % Capacity。容量改变后，同一个键可能属于另一个桶，因此不能只复制桶数组，必须重新计算每个条目的桶下标。

扩容是 O(n)，但不是每次插入都发生，所以插入的平均摊还复杂度仍为 O(1)。最坏情况下所有键冲突，操作会退化为 O(n)。

3.4 比较器必须一致

若使用 StringComparer.OrdinalIgnoreCase，"Algorithm" 和 "ALGORITHM" 是同一个键。哈希码和相等判断必须来自同一个比较器，否则相等键可能落入不同桶。

练习：

- 编写总是返回相同哈希码的比较器，验证冲突处理。
- 记录扩容前后每个键的桶位置。
- 阅读本项目开放寻址实现，手算线性探测和墓碑复用，再与链地址法比较缓存局部性和删除成本。

验收标准：能解释“平均 O(1)”为什么不是“最坏 O(1)”。

4. 并查集：动态连通性

并查集支持：

- Find(x)：返回 x 所在集合的代表。
- Union(a, b)：合并两个集合。
- AreConnected(a, b)：判断是否连通。

4.1 路径压缩

Find 找到根以后，把路径上的节点直接指向根：

```text
压缩前：1 -> 2 -> 3 -> 4
压缩后：1 -> 4，2 -> 4，3 -> 4
```

第一次查询承担压缩成本，后续查询变快。

4.2 按大小合并

合并时让较小树挂到较大树的根下，限制树高。路径压缩和按大小合并共同使用时，摊还复杂度是 O(α(n))，现实规模下非常接近 O(1)。

典型用途：Kruskal、无向图连通分量、账户合并、等价关系分组。

练习：

- 为每个集合维护最小元素或权重和。
- 使用并查集计算二维网格岛屿数量。
- 验证 SetCount 每次有效 Union 恰好减少 1。

5. Trie：以前缀作为路径

插入 car 和 card 后，car 节点既是完整单词，又是 card 的前缀。因此必须使用 IsWord 区分“单词结束”和“仍有孩子”。

删除 car 时不能删除仍被 card 使用的节点。递归返回时，只有节点同时满足以下条件才可以清理：

- 它不代表完整单词。
- 它没有任何孩子。

自动补全先定位前缀节点，再 DFS 收集单词。需要字典序结果时，按字符顺序访问孩子。

Unicode 注意事项

C# 的 char 是 UTF-16 code unit，一个视觉字符可能包含两个代理项或多个组合字符。国际化搜索需要考虑：

- 使用 System.Text.Rune 按 Unicode 标量值遍历。
- 大小写折叠与文化规则。
- Unicode Normalization。
- Ordinal 比较还是文化相关比较。

练习：增加前缀计数、删除数量和最大返回数量；再实现基于 Rune 的版本。

6. LRU：组合数据结构

只用字典可以 O(1) 查找，但不知道最久未使用项；只用链表可以 O(1) 移动和淘汰，但按键查找是 O(n)。组合后：

```text
Dictionary<TKey, LinkedListNode<Entry>> -> O(1) 定位
LinkedList<Entry>                       -> O(1) 移动与淘汰
```

必须维护：

- 字典中的每个节点都在链表中。
- 链表中的每个条目能在字典找到同一个节点。
- 链表头是最近使用，尾部是最久未使用。
- 更新已有键不会增加 Count。
- 淘汰时同时删除字典和链表记录。

教学实现不是线程安全的。生产缓存还要考虑并发、过期时间、缓存击穿、异步工厂、取消和 IDisposable 值的释放。

练习：用一个简单但较慢的参考模型生成随机操作序列，与 LRU 做状态机差分测试。

7. Fenwick 树与线段树

7.1 如何选择

需求	前缀和数组	Fenwick 树	线段树
静态区间和	查询 O(1)	查询 O(log n)	查询 O(log n)
单点更新	O(n)	O(log n)	O(log n)
区间最小/最大	不直接支持	不适合	O(log n)
通用结合操作	否	有限	是
实现复杂度	低	中	较高

数据不更新时，普通前缀和通常最好。高级结构不是默认答案。

7.2 Fenwick 的 lowbit

内部使用 1 基下标，i & -i 得到最低有效位，决定节点覆盖的区间长度：

```text
i = 12 = 1100□
lowbit = 0100□ = 4
```

更新用 i += lowbit(i) 移向包含当前位置的更大区间；查询用 i -= lowbit(i) 拆解前缀。

7.3 线段树的结合律

通用 combine 必须满足：

```text
combine(combine(a, b), c) == combine(a, combine(b, c))
```

加法、最小值、最大值和字符串连接满足结合律；减法不满足。交换律不是必须的，本项目分别维护左右累积结果，所以字符串连接仍保持顺序。

7.4 半开区间

区间 API 使用 [start, endExclusive)：长度是 endExclusive - start，空区间是 [x, x)，相邻区间不会重叠，并与 Span/Range 的切片思维一致。

练习：

- 用直接循环作为参考模型，随机验证更新和查询。
- 在线段树上实现区间最小值。
- 阅读现有 LazyRangeSumSegmentTree，手算懒标记下推；再扩展为区间赋值或区间最小值，解释多个标记如何组合。

8. 图算法选择

最短路最小化某个源点到目标点的路径长度；最小生成树最小化连接全部顶点所选边的总权重。两者不是同一个问题。

算法	图要求	结果	复杂度
BFS	无权或等权	单源最短边数	O(V + E)
Dijkstra	非负权	单源最短路	O((V + E) log V)
Bellman-Ford	可有负权	单源最短路、负环检测	O(VE)
Floyd-Warshall	可有负权、无负环	所有点对最短路	$O(V^3)$
Kruskal	无向图	最小生成森林	O(E log E)
拓扑排序	有向无环图	依赖顺序	O(V + E)

8.1 Bellman-Ford

第 k 轮松弛后，得到最多使用 k 条边的最短路径。简单路径最多使用 V-1 条边，因此执行 V-1 轮足够。

若之后仍能缩短，说明存在从源点可达的负权环。沿负环反复行走会让路径无限下降，所以最短路径不存在。

8.2 Kruskal

按权重从小到大检查边：

1. 两端属于不同连通分量，加入并 Union。
2. 两端已经连通，加入会形成环，跳过。

图不连通时不存在单棵生成树，本项目明确返回最小生成森林和 ComponentCount。

练习：阅读现有 Prim 与 Floyd-Warshall；用随机连通图把 Prim 和 Kruskal 做差分，再让 Bellman-Ford 返回一条实际负环。

9. 进阶动态规划

9.1 编辑距离

状态：

```text
dp[i, j] = source 前 i 个字符变成 target 前 j 个字符的最少操作数
```

转移包括：

- 删除：dp[i - 1, j] + 1
- 插入：dp[i, j - 1] + 1
- 替换或保持：dp[i - 1, j - 1] + cost

当前行只依赖上一行和当前行左侧，所以可以压缩到 O(min(n,m)) 空间。代价是无法直接恢复具体编辑步骤。

9.2 最长递增子序列

O(n²) DP 易于理解。进阶版本维护每个长度对应的最小结尾，并通过二分查找降到 O(n log n)。

最小结尾数组不一定就是实际 LIS。恢复结果需要保存每个长度当前结尾的原始下标，以及每个元素的前驱下标。

动态规划自检：

- 状态是否足以描述未来决策？
- 转移是否覆盖所有选择？
- 初始化是否正确描述空输入？
- 填表方向是否保证依赖已计算？
- 一维压缩是否错误复用了本轮状态？
- 只返回最优值，还是需要恢复具体方案？

10. 字符串算法进阶

算法	核心思想	适用场景
朴素匹配	每个起点逐字符比较	小输入、教学基线
KMP	利用模式串前后缀避免文本回退	单模式精确匹配
Rabin-Karp	滚动哈希比较窗口	多模式、重复窗口、二维扩展
Z Algorithm	计算每个位置与整串前缀的最长匹配	前缀匹配、周期、模式拼接

Rabin-Karp 中哈希相等只表示“可能匹配”。不同字符串可能哈希冲突，必须继续比较原字符：

```csharp
text.AsSpan(start, pattern.Length).SequenceEqual(pattern.AsSpan())
```

Z Algorithm 维护最靠右的前缀匹配区间 [left, right)。新位置在区间内时可以复用已有 Z 值，再从右边界继续比较。

练习：使用 pattern + separator + text 和 Z 数组寻找全部匹配，并说明分隔符必须满足什么条件。

11. 数组、贪心和非比较排序

11.1 Two Sum

字典保存已经出现的值到下标。当前值 x 查找 target - x。先查再加入，保证不会重复使用同一个位置。

11.2 单调队列

滑动窗口最大值保存下标，并保证对应值单调递减：

- 队头过期就移除。
- 新值不小于队尾值时，旧队尾不可能再成为未来最大值。
- 队头始终是当前窗口最大值。

必须保存下标而不是只保存值，才能判断过期并处理重复值。

11.3 Kadane

若当前累计和为负，它只会拖累后续区间，应从当前位置重新开始。不能把最佳值初始化为 0，否则全负数组会错误返回空区间。

11.4 区间调度

每次选择结束最早的可行区间。正确性可以用交换论证：任意最优解的第一个区间都可以替换为结束更早的贪心选择，而不会减少后续选择数量。

贪心不是“选择看起来最好”，必须证明局部选择能扩展为全局最优。

11.5 计数排序

复杂度是 O(n + k)，k 是值域宽度。两个整数若分别接近 int.MinValue 和 int.MaxValue，计数数组仍会巨大。因此 API 必须检查值域，而不能只看元素数量。

12. 组合搜索

括号生成的状态是已放左、右括号数量：

- 左括号未达到 pairCount 时可以添加。
- 只有 closeCount < openCount 时才能添加右括号。

构造中剪枝比生成全部字符串后过滤更高效。

子集问题中，每个位置有“不选”和“选”两条分支，n 个元素产生 2^n 个子集。输出本身就是指数规模，无法通过算法技巧消除。

练习：实现去重排列、组合总和、数独和单词搜索；每次递归返回前检查状态是否完全恢复。

13. 更深入的 C# 15 实践

13.1 API 与所有权

- IEnumerable<T>：只承诺可枚举，可能延迟执行或只能枚举一次。
- IReadOnlyList<T>：支持 Count 和随机访问。
- IList<T>：允许原地修改。
- ReadOnlySpan<T>：同步调用期间的连续内存视图，不拥有内存。
- 数组副本：调用方获得独立快照。

签名应使用满足算法需求的最小接口。排序需要写入，使用 IList<T>；LIS 需要随机访问，使用 IReadOnlyList<T>。

13.2 现代语法的适用边界

```csharp
int[] values = [1, 2, 3];
List<int> list = [1, 2, 3];
return [];
return [.. source, newValue];
```

集合表达式依赖清晰的目标类型。若重载同时接受数组和 Span，可能需要明确类型来消除歧义。

主构造函数适合初始化关系简单的小型节点。初始化包含大量验证、条件或资源管理时，普通构造函数更清晰。

record struct 适合小型不可变值，例如 Interval 和边；具有身份、可变链接或较大字段的节点适合 class。

13.3 溢出与容量增长

容量翻倍使用 checked 可以把整数溢出转为明确异常：

```csharp
var newCapacity = checked(oldCapacity * 2);
```

涉及中点使用 left + (right - left) / 2。涉及值域宽度时先转 long，避免 maximum - minimum 在 int 中溢出。

13.4 惰性枚举与快照

yield return 是惰性的，方法调用时不立即遍历。底层集合在枚举期间变化，结果可能改变或抛异常。

LRU 键顺序返回快照，因为它表达某一时刻的状态；哈希表枚举器是惰性的，更接近 BCL 集合行为。

13.5 线程安全

本项目的数据结构默认都不是线程安全的。简单给每个方法加锁也不一定让复合操作原子化。学习并发结构前应掌握锁粒度、内存可见性、Interlocked、不可变快照和 ConcurrentDictionary 的工厂语义。

14. 测试进阶方法

14.1 差分测试

对相同随机输入运行教学实现和参考实现：

- 排序与 OrderBy 比较。
- 哈希表与 Dictionary 比较。
- LRU 与简单列表模型比较。
- Fenwick/线段树与直接循环比较。
- Kruskal 与 Prim 的总权重比较。

随机种子必须固定，失败时输出种子和操作序列。

14.2 状态机测试

生成 Add、Remove、Update、Query、Clear 等命令，每执行一步就与参考模型比较 Count、内容和查询结果。这类测试比只调用一次 API 更容易发现状态损坏。

14.3 不变量验证

- 哈希表：每个 Entry 的桶下标正确，Count 等于条目总数。
- 并查集：沿 Parent 最终到达自指根，不存在其他父指针环。
- LRU：字典数等于链表数，键无重复。
- 线段树：父节点等于两个孩子的 combine。
- 最小生成森林：边数等于 V - ComponentCount 且不成环。

14.4 性能测试

BenchmarkDotNet 基准应使用多个输入规模，分开测量构建与查询，避免把随机数据生成算入目标方法，消费返回值，并同时观察分配量和耗时。

15. 第 1–8 周进阶学习计划

第 1 周：哈希表

- 手算桶分布、冲突和扩容。
- 写固定哈希码测试。
- 与 Dictionary 做随机差分。

第 2 周：并查集与 Kruskal

- 画路径压缩前后的父指针。
- 实现按大小合并。
- 验证森林边数公式。

第 3 周：Trie 与字符串

- 实现前缀查询和安全删除。
- 比较 KMP、Rabin-Karp、Z。
- 研究 Rune 与 Unicode 规范化。

第 4 周：LRU

- 手算访问和淘汰顺序。
- 编写随机状态机测试。
- 设计过期时间扩展。

第 5 周：区间结构

- 先实现静态前缀和。
- 画 Fenwick 的 lowbit 区间。
- 实现通用线段树并做差分测试。

第 6 周：图算法

- 手算 Bellman-Ford 每轮距离。
- 构造可达与不可达负环。
- 实现 Prim 并与 Kruskal 比较。

第 7 周：进阶动态规划

- 写编辑距离二维表再压缩空间。
- 先写 $O(n^2)$ LIS，再升级到 $O(n \log n)$。
- 恢复一条实际 LIS。

第 8 周：数组、贪心与回溯

- 掌握单调队列和 Kadane。
- 写出区间调度交换证明。
- 分析计数排序值域风险。
- 比较生成后过滤与构造中剪枝。

16. 第 1–8 周核心阶段验收

完成前 8 周后应能够：

- 解释哈希冲突、负载因子和重新散列。
- 实现路径压缩与按大小合并。
- 正确删除 Trie 中共享前缀的单词。
- 用字典和双向链表实现 O(1) LRU。
- 根据更新与查询需求选择前缀和、Fenwick 或线段树。
- 根据权重条件选择 BFS、Dijkstra 或 Bellman-Ford。
- 解释 Kruskal 为什么需要并查集。
- 写出编辑距离状态转移并恢复一条 LIS。
- 解释 Rabin-Karp 为什么哈希相同后仍要比较字符。
- 写出单调队列、Kadane、区间调度和计数排序的适用边界。
- 用泛型、比较器、记录类型、集合表达式和异常契约设计 API。
- 为状态型结构设计随机差分测试和不变量验证。

真正的进阶不是记住更多模板，而是能根据操作集合、输入约束和正确性不变量推导出合适结构。

17. 四个正确性修复：为什么原实现会出错

17.1 Huffman 编码为什么不能返回整数

旧 API 返回 uint。这不是单纯的容量问题，而是表示模型错误：Huffman 码是“带长度的位序列”，整数只保存数值。

```text
位串 0、00、000 -> 整数都等于 0
位串 00101    -> 整数等于 5，长度和两个前导零丢失
```

因此 CalculateHuffmanCode 现在返回 Dictionary<T, string>。字符串直接表达位和长度，编码深度也不再受 32 位限制。实现仍从叶子沿 Parent 回溯：当前节点是父节点左孩子就记录 0，是右孩子就记录 1。回溯方向与最终编码相反，所以先收集位、最后反转。这里没有在循环头部插入字符，因为 StringBuilder.Insert(0, ...) 会反复移动已有内容，可能把线性过程退化为平方级。

建树使用 PriorityQueue<Node, (Weight, Order)>。Weight 保证每次取出两棵最轻的树，Order 只在权重相同的时候提供稳定顺序。相同权重可能产生多棵形状不同但带权路径长度相同的最优树，所以测试不再绑定某一种左右形状，而是检查：

- 根权重等于全部叶权重之和。
- 每个编码非空且只包含 0/1。
- 任意编码都不是另一编码的前缀。
- Decode(Encode(source)) 恢复原序列。

这种测试验证的是算法契约，不是优先队列碰巧采用的平局顺序。

17.2 AVL 的 Change 为什么必须先验证

直接修改节点值可能破坏 BST 顺序，所以 Change 仍应由“删除旧值 + 插入新值”组成。旧实现的问题是操作顺序：如果先删除旧值，而新值已经存在，随后 Insert 抛异常，树就只完成了一半操作。

现在的顺序是：

1. 确认旧值存在，否则立即失败。
2. 若比较器认为新旧值相等，直接返回。
3. 确认新值不存在，否则立即失败。
4. 前置条件全部满足后才执行 Delete 和 Insert。

这称为先验证、后变更。它不等于数据库事务，但在当前单线程、内存内且 Insert 不会发生其他可恢复失败的契约下，可以保证常见失败不会留下半完成状态。测试在异常后同时检查旧值、新值和树内容，避免只断言“确实抛了异常”。

17.3 树重建为什么要区分空树和非法序列

空的前序与空的中序确实表示空树；长度不同的两条序列不是空树，而是调用错误。把两者都返回 null 会掩盖数据损坏。公共验证器现在依次检查：

- 两条序列长度相同。
- 每条序列内部没有重复节点。
- 两条序列包含相同节点集合。
- 构造过程中每个根在当前中序区间内。

唯一性要求很重要。若值重复，仅凭“前序 + 中序”或“后序 + 中序”通常不能唯一确定树，字典也会覆盖重复键的位置。递归解法用中序位置字典把查根从 O(n) 降为 O(1)，总时间从可能的 $O(n^2)$ 降为 O(n)。迭代解法使用显式栈模拟回溯，并在结束时确认整条中序序列被完全消费，借此拒绝“长度和集合正确但左右分区矛盾”的输入。

泛型约束也从 IComparable<T> 收紧为 notnull。算法只需要相等性，不比较大小；不应要求调用方实现没有被使用的能力。

17.4 线索化后为什么还要记录遍历类型

同一空指针在前序线索中可能指向 A，在中序线索中可能指向 B。若结构不记录当前是哪一种线索，遍历方法无法安全地解释指针。CurrentThreadingOrder 现在显式保存该状态；重新线索化会先只沿 Tag == 0 的真实孩子收集节点、清除旧线索，再按新顺序建立前驱和后继。

三种无栈遍历分别利用不同规则：

- 前序：有真实左孩子就向左，否则右指针无论是右孩子还是后继线索都可直接跟随。
- 中序：先到最左节点；右指针是线索就直接到后继，否则进入右子树的最左节点。
- 后序：先下降到最先访问的叶子；后继线索可直接跟随，没有线索时借 Parent 判断应进入父节点右子树还是回到父节点。

后序比前两种更难，是因为访问完孩子后必须知道父节点。为节点增加 Parent 将这部分信息显式化，使匹配当前线索类型的前、中、后序遍历都只需 O(1) 额外空间。层序遍历仍需要队列保存同层候选，空间为 O(w)；不能为了宣传“线索树无栈”而隐藏这一事实。

18. 双向链表与数组双端队列

18.1 双向链表为什么多保存 Previous

单向链表即使维护 Tail，删除尾节点仍要从 Head 找到倒数第二个节点，因此是 O(n)。双向链表让尾节点直接知道前驱，RemoveLast 变成 O(1)，代价是每个节点多一个引用，插入删除时也必须同时维护两个方向。

删除中间节点的核心连接为：

```text
current.Previous.Next = current.Next
current.Next.Previous = current.Previous
```

头尾是特殊边界，所以实现把它们分别复用到 RemoveFirst 和 RemoveLast，避免在多个位置重复修改 Count 与头尾指针。删除后主动断开节点引用不是 GC 的必要条件，但能让调试器中的“节点已经离表”更清楚。

18.2 双端队列为什么使用循环数组

普通数组若每次在下标 0 插入都搬移全部元素，头部操作是 O(n)。循环数组只移动 _head：逻辑下标 i 的物理位置是

```text
(_head + i) % Capacity
```

数组尾部不够不代表没有空间，空槽可能位于数组开头，这就是“绕回”。只有 Count 等于 Capacity 时才扩容。扩容必须按逻辑顺序复制到新数组的 [0, Count)，再令 _head = 0；直接复制物理数组会把绕回后的两段顺序颠倒。

数组双端队列拥有更好的缓存局部性，链式双端队列则无需整体扩容。选择取决于是否重视连续内存、容量波动和节点引用稳定性。

19. 开放寻址哈希表：Empty、Occupied、Deleted 三态

线性探测从初始桶开始依次检查后续槽位，并在数组末尾取模绕回。槽位不能只有“有值/无值”两态。假设 A、B、C 冲突后连续排列，删除 B 若直接设为 Empty，查找 C 会在 B 的位置误以为探测链结束。

因此槽位状态为：

- Empty：从未使用，查找可以终止。
- Occupied：比较键，相等则命中，不等继续。
- Deleted：旧条目的墓碑；查找必须继续，插入可以优先复用。

实现同时维护 Count 和 _usedSlots。Count 只统计真实条目；usedSlots 还包括墓碑。若只根据 Count 判断负载，大量删除会让探测链越来越长却永不整理。usedSlots 超过 0.70 时：真实条目也多就扩容；主要是墓碑多就使用相同容量重散列，以较低成本清理墓碑。

索引使用 (uint)hashCode % capacity，而不是 Math.Abs(hashCode)。Math.Abs(int.MinValue) 仍会溢出，转成 uint 则能无分支地得到合法非负余数。

20. 桶排序与基数排序

20.1 桶排序为什么不是“永远 O(n)”

桶排序先根据 (value - min) / (max - min) 把值归一化到 [0, 1]，再映射到桶。最大值会得到 1，乘桶数后恰好越过末尾，所以实现用 Min(index, bucketCount - 1) 压回最后一桶。

数据均匀时，每桶很小，整体接近 O(n + k)；数据集中时，全部元素可能落入同一桶，复杂度退化为桶内 List.Sort 的 O(n log n)。API 拒绝 NaN、正负无穷和溢出的极端跨度，因为这些值无法形成可靠的线性区间映射。默认桶数取 sqrt(n)，只是通用折中，不是所有数据分布的最优参数。

20.2 LSD 基数排序为什么要求稳定

LSD 从最低有效位开始。高位分配时，必须保留低位已经建立的相对顺序，否则前一轮结果会被破坏。本实现每轮按 source 原顺序写入目标数组，通过前缀位置保证稳定。

每轮处理 8 位，共 256 个桶，32 位整数固定执行四轮。value ^ int.MinValue 只翻转最高符号位：所有负数被映射到 uint 的低半区，非负数映射到高半区，且各自内部顺序不变。因此无需把正负数拆开，也不会对 int.MinValue 取绝对值发生溢出。

21. 左倾红黑树：用局部规则维护全局高度界

左倾红黑树把红链接解释为 2-3 树中同一个多键节点的内部连接。四条不变量是：

1. 根节点是黑色。
2. 红节点不能有红孩子。
3. 红链接只向左倾斜。
4. 从任一节点到所有 null 叶子的黑节点数量相同。

插入后的三个局部修复按顺序执行：右红左黑时左旋；连续两个左红时右旋；左右都红时颜色翻转。这些操作不改变中序顺序，旋转负责改变结构，颜色负责保存等价的 2-3 树含义。

删除采用顶向下策略：下降到下一层前，确保将要进入的孩子不是无法直接删除的 2-节点；MoveRedLeft/Right 通过颜色翻转和旋转从兄弟方向借一个红链接。删除后 Balance 恢复左倾形态。

HasValidInvariants 不是业务 API 的必需品，却是学习项目的重要组成：它同时验证严格 BST 区间、红色约束、左倾约束、左右黑高和实际节点数。随机测试每次变更后都调用它，比只看最终排序结果更容易定位第一次破坏结构的操作。

22. 跳表：用概率换取简单的平衡逻辑

跳表第 0 层包含所有值；节点若出现在第 k 层，也必然出现在 0 到 k-1 层。查找从当前最高层开始，能向右且不会越过目标就移动，否则下降一层。这个过程与在高速公路和辅路之间切换类似。

新节点从 1 层开始，每次以概率 p 晋升，直到失败或达到 maximumLevel。p=0.5 时高度至少为 k 的概率约为 1/2^(k-1)，所以高层节点指数减少，期望搜索路径为 O(log n)。最坏情况下所有节点都只有一层，仍会退化到 O(n)，这与红黑树确定性的最坏 O(log n) 不同。

插入和删除都维护 update[]：它记录目标在每一层的直接前驱。插入用它一次性接入新节点的所有层，删除用它绕过目标。固定 randomSeed 不是算法要求，而是测试最佳实践：让层高选择可复现，失败不会成为难以重现的偶发现象。

23. CI 与测试为什么这样组织

.github/workflows/ci.yml 在 master/main 的 push 与 pull request 上运行：

1. Checkout 仓库。
2. 安装 .NET 11 SDK。
3. Restore，单独暴露依赖问题。
4. Release Build，并使用 --no-restore 避免重复恢复。
5. Test，并使用 --no-build 确保测试的是刚刚构建的产物，同时收集 TRX 与 XPlat Coverage。
6. ReportGenerator 把 Cobertura 数据转换为可读 HTML 和 GitHub Step Summary。
7. upload-artifact 即使测试失败也保留 TRX 与覆盖率文件，便于离线定位。

工作流只授予 contents: read，因为构建测试不需要写仓库或发布包。timeout-minutes 防止异常挂起无限占用 runner。

本轮测试采用六种层次：

- 示例测试：固定小输入，便于手算。
- 边界测试：空结构、单元素、重复值、完整 int 值域。
- 异常测试：非法权重、无效遍历序列、NaN、空结构删除。
- 不变量/差分测试：红黑树逐步验证，跳表与 SortedSet 比较，哈希表强制冲突。
- 性质测试：FsCheck 自动生成并缩减输入，验证排序与红黑树对任意样本都满足规则。
- 状态机测试：每一步都与 Dictionary、LinkedList、SortedSet 对照，而不只比较最终结果。

.github/workflows/mutation.yml 不放进每个 PR 的快速门禁，而是每周或手动运行。原因是 Stryker 会反复修改源码并重跑测试，成本远高于普通测试。当前以 Basic 级别和 4 路并发限定 B+ 树、懒标记区间树、BK-tree、Bloom Filter、LFU 与最小费用流六个新增关键文件；如果存活变异过多，应先补能杀死这些变异的测试，而不是直接扩大范围。2026-07-18 的本地完整运行测试 334 个有效变异，分数为 81.07%，高于 60% break 阈值。

新增算法时不要只提高“测试数量”。更重要的是让每个测试说明一个可能被破坏的契约，并保证失败信息能指出是哪条规则失效。

24. 第 9–12 周学习安排

第 9 周：两端结构与两种哈希策略

- 手算双向链表删除头、尾、中间节点的四个引用变化。
- 画出双端队列绕回和扩容前后的物理数组。
- 用固定哈希码制造冲突，比较链地址节点分配与开放寻址缓存局部性。

第 10 周：非比较排序

- 对同一整数数据分别运行计数和基数排序，写出每轮状态。
- 改变桶数量和数据分布，解释退化原因。
- 说明为什么 NaN 让通常的全序假设复杂化。

第 11 周：红黑树

- 手画左旋、右旋和颜色翻转前后的中序序列与黑高。
- 逐步删除叶子、单孩子节点和双孩子节点。
- 隐藏实现，只根据不变量验证器和测试重写 Add。

第 12 周：跳表与综合验证

- 固定随机种子画出每层索引。
- 用同一命令序列驱动 SkipList 与 SortedSet。
- 比较 AVL、红黑树和跳表的平衡保证、实现复杂度、内存布局和适用场景。

25. 可选算法追踪与 Demo 为什么这样设计

源码入口是 Diagnostics/AlgorithmTracing.cs，演示入口是 DataStructureAndAlgorithm.Demo/Program.cs。追踪能力没有直接使用 Console.WriteLine，因为输出到控制台会把算法与运行环境绑定，单元测试、GUI、Web API 和日志系统都难以复用。当前设计只依赖一个最小接口：

CSHARP

```
public interface IAlgorithmTraceSink
{
    void Record(
        string algorithm,
        string operation,
        string description,
        IReadOnlyDictionary<string, string>? state = null);
}
```

25.1 为什么参数可空

Dijkstra、基数排序和 Huffman 把 IAlgorithmTraceSink? trace = null 放在末尾，红黑树在构造时可选注入。这样做有三个目的：

- 旧调用代码无需修改，属于向后兼容的附加能力。
- 普通执行只进行一次空值判断，不创建事件列表和状态快照。
- 需要教学动画或 JSON 时，同一算法实现可以注入不同 sink，不复制算法。

红黑树把 sink 放在构造函数，是因为旋转和颜色翻转发生在多个私有递归方法中；把依赖保存为只读字段比层层传参更清楚。Dijkstra 和排序是单次静态调用，参数传入更符合生命周期。

25.2 为什么事件是结构化数据

AlgorithmTraceEvent 同时保存递增 Step、Algorithm、Operation、Description 和 State。Description 回答“为什么做”，State 保存本步关键值。若只保存一句拼接后的文本，UI 就无法可靠筛选 Relax、RotateLeft 或 Merge，也难以把节点和距离做成动画。

Demo 只做四件事：构造小输入、注入 CollectingAlgorithmTraceSink、调用公开 API、把最终结果和事件序列化为 JSON。算法库不知道 JSON 的存在，这保持了依赖方向：展示层依赖算法层，算法层不依赖展示层。

25.3 阅读练习

1. 运行 dotnet run --project DataStructureAndAlgorithm.Demo -- radix，解释四次 Pass 为什么稳定。
2. 运行 -- dijkstra，找出某个顶点被松弛两次的原因。
3. 运行 -- red-black，逐步画出 Rotate 与 FlipColors 前后的 2-3 树等价形态。
4. 自己实现一个 Mermaid sink，但不要修改任何算法签名。

26. 从示例测试到性质、状态机和变异测试

不同测试技术回答不同问题，不能互相替代：

技术	主要问题	当前例子	不能证明什么
示例/边界测试	已知场景是否正确	banana 后缀数组、CLRS 最大流	没覆盖的输入是否正确
FsCheck 性质测试	大量生成输入是否都满足通用规则	RadixSort 等于 Array.Sort	性质本身是否足够完整
状态机差分	长操作序列中哪一步状态分叉	Deque 对 LinkedList	参考模型自身没有错误
不变量验证	内部结构是否仍合法	红黑树黑高、索引堆映射	业务输出是否完全正确
覆盖率	哪些代码从未被执行	Cobertura HTML	执行过的断言是否有效
变异测试	测试能否发现人为注入的小错误	条件翻转、算术替换	未变异到的缺陷是否不存在
BenchmarkDotNet	时间与分配的统计差异	Radix、红黑树、哈希表	算法正确性

性质测试中的排序断言使用标准库结果作为 oracle；红黑树性质同时检查有序去重结果、Count 和所有内部不变量。FsCheck 失败后会“缩减”随机数组，例如把数百项反例缩成几个关键值，这比只报告随机种子更容易理解。

状态机测试固定随机种子，并在每一步比较。开放寻址哈希表会经历覆盖、墓碑、重散列；双端队列会经历两端操作、绕回和扩容；有序集合会经历重复插入和不存在删除。若只比较第 2000 步的最终内容，第一次损坏状态的位置会被隐藏。

27. 索引优先队列：为什么需要键到下标的反向索引

普通二叉堆能 O(log n) 入队和出队，但不知道某个 key 当前位于数组哪个位置。要实现 decrease-key，如果线性查找 key，单次更新会退化为 O(n)。IndexedPriorityQueue<TKey, TPriority> 同时维护：

- _heap[index]：按完全二叉树排列的 key 与 priority。
- _indexes[key]：同一个 key 在 _heap 中的下标。

每次 Swap 必须同时更新两个 key 的字典下标；出队时先删除根 key，再把末尾元素移到根并下沉。HasValidInvariants 同时验证堆序、数量和双向映射，因此能发现“堆看起来有序但字典仍指向旧下标”的隐蔽错误。

API 叫 EnqueueOrDecrease 而不是含糊的 Update：已存在 key 只有新优先级更小时才改变。Prim 与 A* 需要的正是这一单向语义；拒绝无意义的 increase 可以减少调用方误用，也让修复方向只有 SiftUp。

28. 六个图算法应该怎样选择

28.1 Prim 与 Kruskal

两者都求无向图最小生成森林，但观察角度不同：

- Kruskal 按全局边权排序，用并查集拒绝成环边，成本主要是 O(E log E)。
- Prim 从一个分量向外扩张，为每个未访问顶点维护当前最轻连接边，索引堆版本为 O((V+E) log V)。

实现外层遍历每个未访问起点，所以断开图返回森林并准确增加 ComponentCount。Prim 可以接受负边，因为生成树只比较边权，不进行“距离一旦确定就不再变”的最短路推理。

测试让 Prim 与已经验证的 Kruskal 比较总权重。这种算法间差分比绑定某一组边更稳健，因为同权图可能有多棵都正确的最小生成树。

28.2 Floyd-Warshall

distances[i,j] 表示当前允许的中转点集合下 i 到 j 的最短距离。加入 k 后只有两种可能：不用 k，或走 i -> k -> j。三重循环正是这个状态转移。

代码先跳过 i -> k 为正无穷的情况，既少做无意义计算，也避免把“不可达 + 负数”误读成有效路径。next[i,j] 保存第一跳而不是完整路径，每次改进时继承 next[i,k]，所以空间仍为 O(V²)。全部中转完成后若 distances[v,v] < 0，说明存在能不断降低代价的负环，最短距离没有定义，必须抛异常。

28.3 Tarjan 强连通分量

discovery[v] 是首次访问时间，low[v] 是 v 的 DFS 子树通过树边和仍在当前栈中的返祖边能够到达的最早时间。只有 low[v] == discovery[v] 时，v 才是一个强连通分量的根，可以弹栈直到 v。

关键细节是 onStack：指向已经完成的另一个分量的边不能降低 low 值。省略它会把本应分开的分量错误合并。整个过程每个顶点入栈出栈一次，每条边检查一次，因此为 O(V+E)。

28.4 桥与割点

无向图同样使用 discovery/low，但判定不同：

- low[child] > discovery[parent]：child 子树没有替代路线回到 parent 或更早祖先，所以 parent-child 是桥。
- 非根节点存在 low[child] >= discovery[parent]：删除 parent 会让 child 子树断开，所以 parent 是割点。
- DFS 根只有至少两个独立子树时才是割点。

无向边在邻接表中出现两个方向，递归时必须跳过父边，否则会立即把它当返祖边，导致所有桥消失。

28.5 A*

A* 的队列优先级是 f(n) = g(n) + h(n)：g 是已经支付的真实代价，h 是对剩余代价的估计。h 恒为 0 时就是 Dijkstra；h 越接近真实剩余距离，通常探索越少。

当前 API 强制 h 有限且非负，边权也必须非负。要保证结果最优，还需要调用方提供不高估真实距离的可采纳启发函数；这无法由通用代码自动证明，所以必须写进契约。代码仍保存 g 到 Distances，不能把 f 当真实距离返回。

28.6 Dinic 最大流

容量网络添加每条正向边时，同时添加容量为 0 的反向残量边，并保存 ReverseIndex。发送 x 单位流后，正向残量减 x，反向残量加 x，这使后续搜索能够撤销早先并不理想的选择。

Dinic 每轮先 BFS 构造只向下一层前进的分层图，再用 DFS 发送阻塞流。nextEdge 记录每个顶点已经证明无用的边，避免同一轮重复扫描。算法在 FlowNetwork.Clone() 上修改残量容量，所以连续两次求流得到相同结果，调用方的原始网络仍可复用。容量网络拒绝自环：自环不会增加 s-t 流量，却会让正向/反向残量索引产生无意义的特殊分支。

29. 有序集合统一契约与比较器最佳实践

IOrderedSet<T> 只抽取红黑树与跳表真正共享的能力：Count、Add、Remove、Contains、Clear、有序枚举、Comparer 和不变量验证。没有把 Height、CurrentLevel、Rotate 等实现细节硬塞进接口，因为统一接口的目标是允许替换，不是抹平结构差异。

与 SortedSet<T> 一样，Comparer.Compare(x, y) == 0 同时表示排序相等和集合重复。若使用 StringComparer.OrdinalIgnoreCase，加入 "Alpha" 后再加入 "ALPHA" 必须返回 false。不能一边用 comparer 排序，一边用 EqualityComparer<T>.Default 去重，否则集合会出现两个按顺序“相同”的元素，查找边界也不再唯一。

遗留 BST/AVL 仍保留 IComparable<T> 约束，避免一次学习扩展引入破坏性 API 变更。新代码优先使用构造函数注入 IComparer<T>；若未来迁移旧树，应先提供兼容重载、契约测试和迁移说明。

30. Aho-Corasick：失败指针为什么能一次扫描多个模式

Trie 共享所有模式的前缀，但普通 Trie 在某条路径失配后不知道文本中已经读过的后缀还能匹配什么。AC 自动机为每个节点增加 Failure：它指向“当前字符串的最长真后缀，同时也是 Trie 前缀”的节点。

构建失败指针使用 BFS，因为父节点的 Failure 必须先完成，子节点才能沿父 Failure 继续寻找。节点还继承 Failure 节点的 Outputs：例如扫描到 she 时，既命中 she，也应命中它的后缀模式 he。

查找时文本下标永不回退。失配只沿 Failure 跳转，所有跳转的摊还成本与文本长度同阶，因此总成本是 O(文本长度 + 命中数量)。构造函数去除重复模式，明确拒绝空模式；空模式会在每个边界命中，不适合这个返回全部命中的教学 API。

31. 后缀数组与 LCP：为什么采用排名倍增

后缀数组保存所有非空后缀起点按字典序排列的顺序。直接比较两个后缀可能逐字符扫描 O(n)，对 n 个后缀排序会很慢。倍增法第 k 轮已知长度 width 片段的排名，只需比较两个整数对：

```text
(rank[start], rank[start + width])
```

这等价于比较长度 2 * width 的前缀。每轮 width 翻倍，直到所有排名唯一。当前教学实现使用 Array.Sort 比较整数对，易读复杂度 O(n log² n)；继续优化可用计数/基数排序把每轮降到 O(n)，总体 O(n log n)。

LCP 使用 Kasai：按原起点顺序访问后缀，若当前后缀与前一个字典序后缀有 common 个相同字符，那么下一个起点至少还能复用 common-1 个。每个字符最多被增加和减少常数次，因此 LCP 构建 O(n)。

FindAll 对排序后的后缀做两次二分，找到以非空 pattern 开头的连续区间。结果对象用只读包装器保护内部后缀数组，调用方不能破坏下一次二分所依赖的排序不变量。返回顺序是后缀字典序，不是文本位置顺序；如果业务需要从左到右展示，应由调用方对结果再排序。

32. Sparse Table 与懒标记线段树的选择边界

32.1 Sparse Table

第 k 层保存每个起点长度 2^k 的聚合值。查询长度 L 时取 k=floor(log2 L)，用左端和右端两个长度 2^k 的块覆盖区间。两个块可能重叠，所以 combine 除了结合律还必须满足幂等律 f(x,x)=x。

Min、Max、GCD 可以使用；Sum 不可以，因为重叠部分会被计算两次。稀疏表预处理 O(n log n)、查询 O(1)，但不支持更新，适合数组固定且查询很多的场景。

32.2 懒标记线段树

LazyRangeSumSegmentTree 的 _sums[node] 始终是节点完整区间的真实和；_lazy[node] 表示这个增量已经作用于当前 sum，但尚未传播到孩子。完全覆盖时用 delta * length 更新 sum 并累积 lazy，不访问叶子；部分覆盖前调用 Push，把标记分给两个孩子，然后递归并 Pull 父和。

区间统一使用半开形式 [start, endExclusive)，空区间加法是无操作、空区间求和为 0。checked 放在和与标记更新处，教学代码选择明确抛出 OverflowException，而不是静默回绕得到看似有效的错误答案。

两者选择可以概括为：数据不变、幂等查询很多时选 Sparse Table；需要区间更新时选懒标记线段树；只有单点更新和前缀和时 Fenwick 更短、更省空间。

33. Span 与 ArrayPool：优化必须连同所有权一起学习

ReadOnlySpan<T> 是连续内存的借用视图，不拥有数据。BinarySearch 接受 Span 后可以直接查数组切片，不需要 ToArray()；返回未找到时采用与 Array.BinarySearch 相同的 ~insertionIndex，既表达失败又保留插入位置。

池化基数排序只租一个辅助数组，四轮在输入 Span 与租用 Span 之间交替写入。偶数轮输入到 buffer，奇数轮 buffer 回输入；四轮结束结果自然位于调用方 Span，省去最终复制。

ArrayPool 的三个重要契约：

- Rent 可能返回比请求更大的数组，只能使用需要的前缀。
- 数组可能含上一个使用者的数据，不能假定初值为零。
- 必须在 finally 中 Return；包含敏感数据或托管引用时要考虑清零。

为什么没有直接删除普通 RadixSort？池化有租还成本、共享池竞争和更复杂的所有权，小数组未必更快。学习项目同时保留普通版与池化版，并在 BenchmarkDotNet 的同一输入分布下比较时间和分配；只有数据证明收益且生命周期契约可控时才采用优化版。

34. 第 13–18 周学习顺序

第 13 周：证据层次与追踪

- 运行四个 Demo，把每个事件对应回源码分支。
- 故意破坏排序的一轮稳定性，观察性质测试给出的最小反例。
- 比较覆盖率 100% 与变异仍存活的差别。

第 14 周：索引堆、Prim 与 A*

- 手算每次 decrease-key 后堆数组和 indexes 字典。
- 让 A* 的 h 恒为 0，与 Dijkstra 做随机图差分。
- 构造一个高估启发函数的反例，说明 API 为什么只能声明、不能自动验证可采纳性。

第 15 周：Floyd 与低链接

- 手写四个顶点的 Floyd 每一层矩阵和 next 第一跳。
- 在同一张图上分别标注 Tarjan SCC、桥和割点，比较 low 值判定为何不同。

第 16 周：最大流

- 画出每条正向边和反向残量边。
- 对 CLRS 网络逐轮画分层图，验证源流出量等于汇流入量。
- 枚举小图 s-t 割，与 Dinic 最大流做差分。

第 17 周：多模式与后缀索引

- 为 he/she/hers 手画 Trie、Failure 和 Outputs。
- 对 banana 写出全部后缀、排序、排名倍增和 LCP。
- 比较 AC 适合“很多模式扫一段文本”，后缀数组适合“一个固定文本做很多查询”。

第 18 周：区间与内存

- 用 Min 构建 Sparse Table，标出查询的两个重叠块，并说明 Sum 为什么错误。
- 画懒标记覆盖、Push、Pull 的状态。
- 正式运行 SortingBenchmarks，记录普通版与池化版的 Mean、Allocated，并解释是否值得采用。

完成这六周后，不应只会背算法名字，而应能从数据是否更新、查询次数、图边权、所需输出、内存所有权和验证成本推导出选择。

第三篇 API 设计与比较器约定

> 统一空值、区间、比较器、所有权、数值和验证契约，说明现代 C# 写法为什么服务于可预测的 API。

title: API 设计与比较器约定 description: 统一参数验证、半开区间、比较器、集合重复、输入所有权与异常类型约定。

本文件是新增数据结构与算法的统一契约。目标不是让所有旧类型一次性改成同一种外观，而是让新 API 可预测，并在迁移旧代码时保持兼容。

1. 参数验证

- 公共引用参数使用 ArgumentNullException.ThrowIfNull，让参数名准确进入异常。
- 数字必须满足范围时使用 ArgumentOutOfRangeException，例如容量、概率、权重或启发值。
- 参数类型正确但组合不合法时使用 ArgumentException，例如遍历序列不一致。
- 对象当前状态不允许操作时使用 InvalidOperationException，例如空队列出队、负环导致最短路无定义。
- 查找一个按契约必须存在的键失败时使用 KeyNotFoundException。

为什么区分异常类型：调用方可以区分“参数本身错误”“对象状态错误”和“数据不存在”，测试也能更精确地表达契约。

2. 区间统一为半开形式

公开范围方法使用 [start, endExclusive)：

- 长度直接是 endExclusive - start。
- 空区间是 [x, x)。
- 相邻区间 [a, b) 与 [b, c) 不重叠。
- 与数组切片、Span<T> 和 .NET Range 的思维一致。

只有数学上没有空区间单位元的结构才拒绝空查询。例如 Sparse Table 的 Min 查询拒绝空区间；区间求和的单位元是 0，所以懒标记线段树允许空查询并返回 0。

3. 相等与排序

哈希结构接收 IEqualityComparer<T>；有序结构接收 IComparer<T>。二者不能混用：

- 哈希表要求 Equals 相等的键必须产生相同哈希码。
- 有序集合以 Compare(x, y) == 0 判断重复，与 SortedSet<T> 一致。
- 比较器由构造函数注入并通过只读属性公开，整个对象生命周期内不能替换。

红黑树与跳表实现 IOrderedSet<T>。接口只包含真正可互换的集合能力，不暴露树高、跳表层数等实现细节。

旧 BST/AVL 的 IComparable<T> 约束暂不删除，因为那会破坏调用方泛型约束和构造方式。迁移时应先加入比较器重载、通用契约测试和弃用说明，而不是直接修改公开签名。

4. 重复值与修改返回值

- 集合的 Add/Remove 返回 bool，表示状态是否真的改变。
- 映射的 TryAdd 拒绝覆盖，索引器赋值负责新增或覆盖。
- 图的重复边不是无声无操作：WeightedGraph.AddEdge 抛异常，避免调用方误以为权重已更新。
- 容量网络拒绝自环，因为自环不增加 s-t 流量，保留它只会复杂化成对残量边的索引契约。
- 需要更新边权时应提供语义明确的 UpdateEdge，而不是让 Add 同时承担两种含义。

为什么返回 bool：调用方不必先执行 Contains 再修改，避免重复搜索，也能直接写差分测试。

5. 输入与输出所有权

- 方法若会原地修改输入，名称、参数类型或 XML 注释必须明确说明。
- 只读算法优先接收 IReadOnlyList<T>、IEnumerable<T> 或 ReadOnlySpan<T>。
- 返回集合默认不暴露可修改的内部数组或列表。
- Span<T> 是借用视图，不能跨 async/iterator 生命周期保存。
- 从 ArrayPool<T> 租出的数组必须在 finally 归还；可能含敏感数据时清零。

Dinic 在容量网络的克隆上修改残量边，原因正是原始网络属于调用方；连续求流不应继承上一次的残量状态。

6. 数值契约

- 图权重、容量和启发值拒绝 NaN/Infinity，因为比较和加法无法维持算法不变量。
- Dijkstra/A* 拒绝负边；Prim 可以接受负边；Floyd/Bellman-Ford 接受负边但必须检测负环。
- 累计和使用 long，并在教学实现关键位置使用 checked，让溢出显式失败。
- 浮点残量比较使用小 epsilon，避免多次减法留下极小正数造成无效递归。

7. 可选诊断不污染核心算法

算法追踪通过可选 IAlgorithmTraceSink 注入。没有 sink 时不创建事件；有 sink 时记录结构化操作与状态。算法层不直接写控制台、不序列化 JSON，也不引用 Demo 项目。

不变量方法用于测试和学习，命名为 HasValidInvariants。它们可以是 O(n)，不能让调用方误以为适合每次生产操作后执行。

8. 测试与性能证据

新增模块至少应有：

1. 一个可手算的正常例子。
2. 空输入、单元素、重复值和边界数值。
3. 非法参数与非法状态。
4. 与标准库、朴素模型或另一算法的差分测试。

5 状态型结构的不变量或长操作序列。

6 只有声称性能收益时才加入 BenchmarkDotNet 基准。

覆盖率只说明执行过，变异测试进一步检查断言能否发现逻辑变化；两者都不能取代正确性推导。性能基准只比较成本，也不能证明结果正确。

9. C# 15 写法边界

- 使用集合表达式 []、[a, b] 和扩展元素减少样板代码，但在重载可能歧义时写出显式类型。
- 使用主构造函数表达只负责保存参数的私有 Node/Entry；公共复杂类型仍优先显式构造函数，便于验证参数和写 XML 文档。
- 使用模式匹配表达互斥边界，例如 value is < 0 or > 1。
- 保持可空引用类型启用，不用 ! 隐藏未证明的空值；只有结构不变量已经保证非空的局部位置才使用 null-forgiving。
- 不为追求“新语法”牺牲算法不变量的可见性。清楚的循环和局部变量通常比高度压缩的 LINQ 更适合核心热路径。

第四篇　LeetCode 100 题题型地图

> 按题型说明每道题为什么采用当前方法、需要维护什么不变量，以及复杂度从何而来。

title: LeetCode 100 题 C# 15 完整学习指导 description: 按题型组织 100 道不重复题，逐题解释算法选择、不变量、复杂度与常见错误。

1. 学习目标

本专题在原有 19 道经典题基础上新增 81 道不重复题目，合计正好 100 道。目标不是背诵答案，而是建立“题目约束 → 数据结构 → 不变量 → 复杂度 → C# API”的推导链。

源码目录：DataStructureAndAlgorithm/LeetCode/

测试目录：DataStructureAndAlgorithm.Test/LeetCodeTest/

原有 19 道题的逐题推导见 LeetCode 经典题目 C# 学习指导。本文重点解释新增 81 道题为什么采用当前解法，并给出完整 100 题路线。

2. 完整题号清单

原有核心 19 题

1, 3, 15, 20, 21, 42, 70, 76, 102, 121, 139, 141, 198, 200, 206, 207, 215, 236, 322

新增数组与矩阵 20 题

11, 26, 27, 31, 33, 34, 36, 48, 54, 56, 66, 73, 75, 88, 128, 136, 169, 189, 217, 238

新增字符串 15 题

5, 8, 14, 49, 125, 202, 242, 387, 394, 409, 438, 567, 647, 763, 1768

新增链表与设计 14 题

2, 19, 23, 24, 25, 138, 142, 143, 148, 155, 160, 234, 703, 876

新增二叉树 15 题

98, 103, 104, 108, 114, 124, 199, 226, 230, 235, 297, 437, 543, 572, 1448

新增图与回溯 10 题

39, 46, 79, 133, 695, 785, 797, 841, 994, 1971

新增动态规划、贪心与单调栈 7 题

55, 62, 64, 84, 152, 209, 221

以上集合没有重复题号，共 19 + 20 + 15 + 14 + 15 + 10 + 7 = 100。

3. 数组与矩阵：20 题为什么这样做

题号	方法	为什么采用该方法	关键不变量	复杂度
11	MaximumContainerArea	面积受较短边限制，移动高边只减小宽度	指针外侧组合已不可能更优	O(n) / O(1)
26	RemoveDuplicatesFromSortedArray	有序性让重复元素连续，可用读写指针压缩	[0, write) 是去重后的有效前缀	O(n) / O(1)
27	RemoveElement	只需把保留值稳定写回前缀，不必真实删除	[0, write) 均不等于目标值	O(n) / O(1)
31	NextPermutation	最右下降后缀已经是最大排列，只需替换枢轴并反转	枢轴右侧在修改前非递增	O(n) / O(1)
33	SearchRotatedSortedArray	旋转数组每轮至少一半仍有序，可判断目标在哪侧	搜索区间始终包含所有可能目标	O(log n) / O(1)
34	SearchRange	两次边界二分直接定位重复区间	LowerBound/UpperBound 使用半开区间	O(log n) / O(1)
36	IsValidSudoku	9 个数字可用位掩码表示，检查和写入均 O(1)	每行、列、宫的 bit 只出现一次	O(81) / O(1)
48	RotateImage	转置把行列互换，反转行完成顺时针映射	转置只访问上三角，避免交换两次	$O(n^2)$ / O(1)
54	SpiralOrder	四边界逐层收缩可保证每格只访问一次	边界内是未访问矩形	O(mn) / O(1)
56	MergeIntervals	按起点排序后，新区间只可能与最后结果重叠	结果始终有序且互不重叠	O(n log n) / O(n)
66	PlusOne	十进制进位只需从最低位向左传播	当前下标右侧已经是正确结果	O(n) / O(n)
73	SetMatrixZeroes	首行首列可复用为行列标记，避免额外集合	标记阶段不提前清零内部元素	O(mn) / O(1)
75	SortColors	三种值对应 <1、=1、>1 三个区域	low 左侧全 0，high 右侧全 2	O(n) / O(1)
88	MergeSortedArrays	从尾部写入不会覆盖 first 尚未处理的元素	write 右侧始终是最终最大后缀	O(m+n) / O(1)
128	LongestConsecutiveSequence	只从没有前驱的数扩展，每个数最多被访问两次	每条连续链只从最小值开始	平均 O(n) / O(n)
136	SingleNumber	x ^ x = 0，成对元素全部抵消	result 是已扫描元素的异或值	O(n) / O(1)
169	MajorityElement	多数元素与其他元素抵消后仍会留下候选	votes 表示候选净优势	O(n) / O(1)
189	RotateArray	整体反转再反转两段，得到目标循环顺序	三次反转等价于右旋	O(n) / O(1)
217	ContainsDuplicate	HashSet.Add 失败即可立即确认重复	set 只包含此前元素	平均 O(n) / O(n)
238	ProductExceptSelf	每个答案是左前缀积乘右后缀积，不需要除法	第二轮前 result 已保存左侧积	O(n) / O(n)

3.1 为什么原地题常用读写指针

26、27、75、88 都要求把答案写回原数组。真正删除元素会反复移动数据，可能变成 $O(n^2)$。读指针负责检查，写指针只在确定答案后前进，可以把每个元素的写入次数限制为常数。

以 27 为例：

```text
read 扫描全部元素
write 指向下一个保留位置
[0, write) 永远是已经确认的答案
```

C# 中不能用 foreach 枚举 List<T> 的同时通过索引写入，因为索引 setter 会更新集合版本。本项目显式使用 for，注释说明这个运行时规则。

3.2 为什么矩阵题先验证形状

C# 的 int[][] 是锯齿数组，不保证每行等长，也允许某行是 null。LeetCode 输入通常满足矩形约束，但教学库的公共 API 会先验证，避免算法中出现难以理解的越界异常。

3.3 常见错误

- 31 只交换枢轴却不反转后缀。
- 33 把普通二分的判断直接套在旋转数组上。
- 73 在完成标记前就清零，导致零扩散到不应清零的位置。
- 88 从头合并，覆盖 first 中尚未读取的值。
- 128 对每个数都向两侧扩展，退化为 O(n²)。
- 169 只返回候选却不验证输入确实存在多数元素。

4. 字符串：15 题为什么这样做

题号	方法	为什么采用该方法	关键不变量	复杂度
5	LongestPalindromicSubstring	每个回文由奇数或偶数中心唯一决定	扩展区间始终是回文	O(n²) / O(1)
8	StringToInteger	输入格式有固定状态顺序：空格、符号、数字	index 之前字符已按状态消费	O(n) / O(1)
14	LongestCommonPrefix	候选前缀只会缩短，不会重新增长	前缀对已处理字符串都成立	O(总字符数)
49	GroupAnagrams	排序后的字符序列是异位词的规范键	同组字符串具有相同 key	O(n·k log k)
125	IsPalindromeIgnoringSymbols	两端跳过无关字符后直接比较	指针外侧已经满足回文条件	O(n) / O(1)
202	IsHappyNumber	状态有限，若不到 1 必然进入循环	seen 保存已出现的数字状态	O(log n) 状态
242	IsAnagram	频次相同是异位词的充分必要条件	第二串逐步消耗第一串频次	O(n) / O(字符集)
387	FirstUniqueCharacter	第一次扫描计数，第二次保持原顺序	返回首个频次为 1 的下标	O(n)
394	DecodeString	嵌套编码具有后进先出结构	栈保存每层前缀和重复次数	O(输出长度)
409	LongestPalindromeLength	偶数部分全可用，中心最多放一个奇数	每种字符贡献 count/2*2	O(n)
438	FindAnagramStartIndexes	异位词窗口长度固定，可维护缺失字符数	窗口长度不超过 pattern 长度	O(n)
567	ContainsPermutation	等价于是否存在任一异位词窗口	复用 438 的窗口契约	O(n)
647	CountPalindromicSubstrings	每个回文对应唯一中心和半径	每次扩展都产生一个新回文	O(n²)
763	PartitionLabels	片段必须覆盖片段内所有字符的最后位置	end 是片段字符 last 的最大值	O(n)
1768	MergeAlternately	相同下标依次取两串，剩余部分自然追加	index 前字符均已交替写入	O(n+m)

4.1 为什么 5 和 647 都使用中心扩展

两题的差别只在聚合方式：5 维护最长区间，647 每成功扩展一次就计数。中心扩展比枚举所有子串再反转验证少一层循环，并自然覆盖奇数与偶数长度。

4.2 为什么 438 不每次比较两个字典

若每移动窗口一次就完整比较频次表，会把复杂度乘上字符集大小。项目维护 missing：加入一个仍缺少的字符时减一，移除后重新缺少时加一。missing == 0 就是充分条件。

4.3 C# 字符与 Unicode

这些题遵循 LeetCode 常见的 char 语义。char 是 UTF-16 code unit，不一定等于用户看到的一个字符。真实国际化文本需要考虑 Rune、组合字符、Normalization 和文化相关大小写规则。

5. 链表与设计：14 题为什么这样做

题号	方法/类型	为什么采用该方法	关键不变量	复杂度
2	AddTwoNumbers	位按低位到高位存储，天然顺序传播进位	carry 是上一位产生的进位	O(max(n,m))
19	RemoveNthFromEnd	快指针先走 n 步后，两指针距离恒为 n	fast 到尾时 slow 在前驱	O(n) / O(1)
23	MergeKSortedLists	堆只需比较 k 个当前链表头	堆保存每条非空链表最小未处理节点	O(N log k)
24	SwapPairs	dummy 让头部交换与中间交换使用同一逻辑	beforePair 指向待交换对的前驱	O(n)
25	ReverseNodesInKGroup	先确认完整 k 组，避免反转不足部分	groupAfter 标记本组停止位置	O(n)
138	CopyRandomList	Random 可任意指向，映射保存原节点到副本	两种指针都只连接副本	O(n) / O(n)
142	DetectCycleEntry	Floyd 相遇后的距离关系保证入口再次相遇	第二阶段两指针等速	O(n) / O(1)
143	ReorderList	拆成前后两半，反转后半再交替合并	两条待合并链都保持内部顺序	O(n)
148	SortList	归并排序适合链表，合并无需随机访问	每层把链表分成两个较短链	O(n log n)
155	MinStack	每层同步保存“到这里的最小值”	栈顶元组含当前全局最小值	每操作 O(1)
160	GetIntersectionNode	两指针各走 A+B，总路径长度相同	第二轮消除头部长度差	O(n+m)
234	IsPalindromeList	反转后半段即可 O(1) 空间比较	比较后恢复输入链表	O(n) / O(1)
703	KthLargestStream	大小为 k 的最小堆，堆顶就是第 k 大	堆只保留当前最大 k 个值	每次 O(log k)
876	MiddleNode	快指针速度是慢指针两倍	fast 到尾时 slow 到中点	O(n)

5.1 dummy 节点为什么重要

19、23、24、25 都可能改变链表头。没有 dummy 时，头节点需要单独分支；dummy 把“头前驱”变成真实节点，让删除、接入和交换统一为修改 before.Next。

5.2 为什么查询后要恢复链表

234 的目标是判断而非变换。如果反转后半段后直接返回，调用方得到的链表会被截断或改变。教学实现再次反转恢复，强调公共 API 的副作用契约。

5.3 节点身份与节点值

环入口、链表相交和 Random 指针都依赖节点引用身份。两个值相同的节点并不是同一个节点，因此测试使用 Assert.Same / ReferenceEquals。

6. 二叉树：15 题为什么这样做

题号	方法	为什么采用该方法	递归返回值/不变量	复杂度
98	IsValidBinarySearchTree	只比较父子不够，必须传递整条祖先上下界	节点值位于开放区间 (lower, upper)	O(n)
103	ZigzagLevelOrder	BFS 保证按层，反向下标避免层后再 Reverse	一轮只消费当前 levelSize	O(n)
104	MaximumDepth	树高由较高子树加一直接定义	返回当前子树高度	O(n)

题号	方法	为什么采用该方法	递归返回值/不变量	复杂度
108	SortedArrayToBinarySearchTree	中点作根使左右规模接近	[start,end) 对应当前子树	O(n)
114	FlattenToLinkedList	前序栈明确下一个节点，便于原地重连	previous 是已展开链尾	O(n)
124	MaximumPathSum	父节点只能延伸一侧，但全局路径可同时用两侧	返回可向父延伸的最大收益	O(n)
199	RightSideView	每层最后访问节点即右侧可见节点	index==levelSize-1 时记录	O(n)
226	InvertTree	每个节点局部交换左右子树即可	返回已翻转的当前子树	O(n)
230	KthSmallest	BST 中序序列升序，第 k 个即答案	栈模拟递归左链	O(h+k)
235	LowestCommonAncestorInBst	两目标同侧就继续该侧，否则当前分叉	lower/upper 包含两个目标值	O(h)
297	Serialize/Deserialize	前序值加空节点标记可唯一描述结构	反序列化按完全相同顺序消费 token	O(n)
437	CountPathsWithSum	路径和转为前缀差查询	字典只保存当前根到节点路径	O(n)
543	Diameter	经过节点的最长路径是左右高度之和	返回子树高度，全局更新直径	O(n)
572	IsSubtree	找到同值候选根后必须比较完整结构	AreSame 同时比较值与空节点位置	最坏 O(nm)
1448	CountGoodNodes	好节点只取决于祖先路径最大值	maximum 是当前节点之前的路径最大值	O(n)

6.1 树递归先写“返回值含义”

树题最容易写出能跑但无法证明的递归。动手前先写一句：

- 104 返回“当前子树高度”。
- 124 返回“可以向父节点继续延伸的单支最大和”。
- 543 返回高度，但用全局变量记录左右高度之和。
- 437 返回“当前子树内满足目标的路径数量”。

返回值含义固定后，递归组合通常自然出现。

6.2 297 为什么必须保存空节点

只有前序节点值无法区分“左孩子为空”和“右孩子为空”。# 标记把树形结构编码进序列，使反序列化可以唯一恢复。

7. 图与回溯：10 题为什么这样做

题号	方法	为什么采用该方法	关键状态	复杂度
39	CombinationSum	非递减下标避免相同组合的排列重复	start 限制下一次可选范围	与答案规模相关
46	Permutations	每一层选择一个尚未使用的位置	used 表示当前路径已选位置	O(n·n!)
79	WordExists	单词路径依赖当前路径访问状态，适合回溯	visited 只对当前递归路径有效	O(mn·4^L)
133	CloneGraph	图可能有环，映射同时去重并找到副本	copies 保存原节点→副本	O(V+E)
695	MaximumIslandArea	每个连通分量 DFS 直接累计面积	visited 防止重复计数	O(mn)
785	IsBipartite	二分图等价于相邻节点颜色相反	colors 为 0/1/-1	O(V+E)
797	AllPathsSourceToTarget	需要输出所有路径，回溯维护当前 path	path 是源点到当前节点	与输出规模相关
841	CanVisitAllRooms	钥匙就是图的出边，从 0 做可达性遍历	visited 记录已进入房间	O(V+E)
994	MinutesToRotAllOranges	所有腐烂橘子同时扩散，需要多源 BFS	每层队列代表一分钟边界	O(mn)
1971	HasValidPath	普通无向连通性，邻接表 DFS 足够	visited 是源点可达集合	O(V+E)

7.1 回溯为什么必须撤销

39、46、79、797 共享模板：选择 → 递归 → 撤销。selected、used、visited 和 path 都属于当前递归路径；如果不撤销，兄弟分支会继承不属于自己的状态。

7.2 994 为什么是多源 BFS

若对每个腐烂橘子分别 BFS，会重复扫描。把所有初始腐烂位置同时入队，相当于添加一个虚拟超级源点。每一层统一表示经过一分钟，第一次感染就是最短时间。

7.3 133 为什么不能只按节点值去重

图节点值未必唯一，边连接的是对象身份。副本映射使用节点引用作为键，保证即使多个节点值相同也能正确克隆拓扑。

8. 动态规划、贪心和单调栈：7 题为什么这样做

题号	方法	为什么采用该方法	关键状态	复杂度
55	CanJump	只关心当前前缀最远可达位置，不需枚举所有跳法	farthest 是已处理位置可达最远点	O(n)
62	UniquePaths	到当前格只可能来自上方和左方	dp[column] 更新前是上方，更新后是当前	O(mn) / O(n)
64	MinimumPathSum	最优路径同样只依赖上、左较小代价	costs 保存当前行最优前缀	O(mn) / O(n)
84	LargestRectangleArea	柱子遇到更矮右边界时才能确定最大宽度	栈内高度对应下标单调递增	O(n)
152	MaximumProductSubarray	负数会让最小积变最大积，必须同时维护两者	max/minEnding 是以当前位置结尾的极值	O(n)
209	MinimumSubarrayLength	全为正数使窗口和对边界具有单调性	sum 是 [left,right] 的和	O(n)
221	MaximalSquareArea	右下角边长由上、左、左上最小值决定	dp[column] 是当前格最大边长	O(mn) / O(n)

8.1 55 为什么贪心足够

题目只问能否到达，不问最少跳数或具体路径。对每个可达位置更新最远边界即可；在最远边界内的所有具体跳法对未来的贡献都被一个最大值概括。

8.2 84 为什么要加虚拟零高度

递增到数组结尾的柱子没有自然的更矮右边界。循环末尾加入高度 0 的虚拟柱，可以统一触发所有剩余柱出栈，不需要额外清栈代码。

8.3 209 为什么要求正数

正数保证右边界扩张时 sum 不减，左边界收缩时 sum 不增。如果允许负数，窗口和失去单调性，滑动窗口可能错过答案，应改用前缀和与其他结构。

9. C# 15 实现原则

9.1 输入副作用必须明确

原地题方法名和注释明确说明修改输入。其他方法尽量复制输入，例如 56 复制区间、215 Quickselect 复制数组、994 复制网格。这样测试可以验证调用方数据没有被意外改变。

9.2 用 long 保护中间结果

面积、路径数、乘积和三数之和可能超出 int。即使题目约束保证最终值可表示，中间表达式仍应先提升到 long，避免 C# unchecked 回绕产生看似合法的错误值。

9.3 nullable 节点表达真实状态

链表和树的空节点使用 ListNode? / TreeNode?。! 只在控制流已经证明非空但编译器无法推断时使用，并尽量让相邻代码说明证明条件。

9.4 最小接口表达算法需求

- 只枚举：IEnumerable<T>。
- 需要 Count 和下标：IReadOnlyList<T>。
- 需要原地写入：IList<T>。
- 固定外部题目签名：数组或节点模型。

签名本身就是文档：调用方一眼能看出算法是否需要随机访问或是否会修改集合。

9.5 集合表达式与明确目标类型

C# 15 的 [1,2,3]、[] 和 [.. values] 减少样板代码。但当重载同时接受数组和 Span 时，测试可能出现歧义，此时使用 SequenceEqual 或显式类型，而不是依赖编译器猜测。

10. 测试策略

新增 81 道题均有以题号命名的测试方法，例如 P84_LargestRectangleArea。这样可以从测试列表直接审计题库是否真的达到 100，而不是只统计源码中的方法数量。

每类测试重点：

- 原地数组：验证有效前缀或完整修改结果。
- 字符串：验证空串、重复字符、嵌套格式和 Unicode 相关契约。
- 链表：验证节点身份、输入恢复和环。
- 树：验证结构而不只验证某个值。
- 图：验证有环、断连、多源传播和输入不变性。
- DP：验证边界初始化与空间压缩方向。

进一步练习差分测试：

- 31 与“生成全部排列后排序”的小规模参考实现比较。
- 33/34 与线性扫描比较。
- 56 与逐点覆盖参考模型比较。
- 148 与 OrderBy 比较。
- 84 与枚举所有区间比较。
- 221 与枚举所有正方形比较。

11. 16 周学习顺序

1 第 1–2 周：1、11、26、27、121、217、238。

2 第 3–4 周：3、5、14、20、49、125、242、387。

3 第 5 周：15、31、33、34、56、75、88。

4 第 6 周：42、73、128、136、169、189、215。

5 第 7–8 周：2、19、21、23、24、25、206、876。

6 第 9 周：138、141、142、143、148、155、160、234、703。

7 第 10–11 周：98、102、103、104、108、114、124、199、226。

8 第 12 周：230、235、236、297、437、543、572、1448。

9 第 13 周：39、46、79、133、200、695。

10 第 14 周：207、785、797、841、994、1971。

11 第 15 周：55、62、64、70、139、198、209、221、322。

12 第 16 周：8、36、48、54、66、76、152、394、409、438、567、647、763、1768。

每周都应完成：手算状态、独立实现、运行题号测试、写出不变量、分析复杂度、完成一个变式。

12. 验收标准

完成 100 题后，应能够：

- 根据“有序、连续、连通、层次、依赖、最优前缀”等关键词快速缩小算法范围。
- 在写代码前明确循环或递归不变量。
- 解释为什么双指针可以移动某一侧，而不是仅记住模板。
- 解释为什么某些滑动窗口必须要求正数或固定窗口长度。
- 在链表题中安全保存后继、统一处理头节点并识别引用身份。
- 为树递归写出精确返回值含义。
- 区分 DFS、BFS、多源 BFS、拓扑排序、染色和回溯的状态生命周期。
- 为 DP 写出状态、转移、初始化、遍历顺序和空间压缩依据。
- 在 C# 中处理整数溢出、nullable、锯齿数组、集合枚举版本和输入副作用。
- 用题号测试、边界测试和差分测试证明实现，而不是只通过一个样例。

真正掌握 100 题的标志，是遇到第 101 题时能够重新推导，而不是继续寻找相同代码模板。

13. P3 追加 14 题

原有 100 题继续作为基础题型地图；P3 独立追加 50, 69, 191, 204, 231, 312, 332, 337, 338, 516, 698, 847, 1044, 1192，与上面的 100 个题号没有重复。

- 数值与位运算：50、69、191、204、231、338。
- 区间、树形和状态压缩 DP：312、337、516、698、847。
- 图与路径：332、1192。

- 后缀结构：1044。

这 14 题不用于把题库名称改成另一个整数，而是分别验证 P3 新增的快速幂、筛法、位状态、区间 DP、树形 DP、欧拉路径、低链接和后缀索引。

逐题为什么这样做、核心不变量、复杂度和常见错误见 P3 数论、位运算与高级算法学习指导。

第五篇 核心 19 题精讲

> 选取 19 道高频题展开推理过程，帮助把题型识别落实为可独立书写、可测试的 C# 代码。

title: LeetCode 经典 19 题 C# 学习指导 description: 用 19 道经典题学习数组、字符串、链表、树、图、动态规划与回溯的可迁移模式。

1. 目标与使用方式

本专题不是按题号背答案，而是用经典题学习可迁移的算法模式。题目描述均为学习化概述，源码位于 DataStructureAndAlgorithm/LeetCode/，测试位于 DataStructureAndAlgorithm.Test/LeetCodeTest/。

题库现已扩展到 100 道；完整题号地图、新增 81 题的逐题“为什么”解释和 16 周路线见 LeetCode 100 题完整学习指导。

建议每道题执行五步：

1. 写出暴力解法及其复杂度。
2. 找出重复计算、单调性或可复用状态。
3. 明确循环/递归不变量。
4. 独立写代码，先覆盖最小与异常输入。
5. 对照项目实现，总结可迁移的模板而不是记代码行。

2. 题目地图

题号	题目	模式	项目方法	复杂度
1	Two Sum	哈希表	ClassicArrayProblems.TwoSum	O(n)
3	Longest Substring Without Repeating Characters	滑动窗口	LongestSubstringWithoutRepeatingCharacters	O(n)
15	3Sum	排序、双指针、去重	ThreeSum	$O(n^2)$
20	Valid Parentheses	栈	HasValidParentheses	O(n)
21	Merge Two Sorted Lists	链表双指针	MergeTwoSortedLists	O(n + m)
42	Trapping Rain Water	左右双指针	TrapRainWater	O(n)
70	Climbing Stairs	一维动态规划	ClimbStairs	O(n)
76	Minimum Window Substring	可变滑动窗口	MinimumWindowSubstring	O(n + m)
102	Binary Tree Level Order Traversal	BFS	ClassicTreeProblems.LevelOrder	O(n)
121	Best Time to Buy and Sell Stock	前缀最优状态	MaximumStockProfit	O(n)
139	Word Break	前缀 DP	WordBreak	最坏 $O(n^2)$
141	Linked List Cycle	快慢指针	HasCycle	O(n)
198	House Robber	状态压缩 DP	HouseRobber	O(n)
200	Number of Islands	网格 DFS	NumberOfIslands	O(rows × columns)
206	Reverse Linked List	指针重连	ReverseList	O(n)
207	Course Schedule	拓扑排序	CanFinishCourses	O(V + E)

题号	题目	模式	项目方法	复杂度
215	Kth Largest Element	Quickselect	FindKthLargest	平均 O(n)
236	Lowest Common Ancestor	树递归	LowestCommonAncestor	O(n)
322	Coin Change	完全背包 DP	CoinChange	O(amount × coinCount)

本章精讲最初的 19 道核心题；项目现有 100 个不重复题号。部分算法与通用实现呼应，例如 322 通过适配器复用 MinimumCoins，展示“刷题 API”和“可复用算法库 API”的区别。

3. 哈希表题型

3.1 LeetCode 1 - Two Sum

目标概述：在整数序列中找到两个不同位置，使两数之和等于目标。

暴力解法枚举所有下标对，时间 O(n²)。优化思路是：扫描到 value 时，不需要回头逐个比较，只要询问此前是否见过 target - value。

不变量：在处理下标 i 前，字典只保存 [0, i) 中已经访问过的值。

```text
numbers = [2, 7, 11, 15], target = 9
i=0, value=2, 查找7失败，记录 2->0
i=1, value=7, 查找2成功，返回 [0,1]
```

为什么先查再加入：如果目标是 6、当前值是 3，先加入会把同一个下标同时作为两个答案。

C# 注意：target - value 可能发生 int 溢出。项目先提升到 long，确认补数仍在 int 范围内再查字典。

常见错误：

- 返回值而不是下标。
- 重复使用同一下标。
- 使用 ContainsValue 导致 O(n²)。
- 忽略整数回绕造成伪匹配。

扩展练习：Two Sum II（有序数组）、返回全部下标对、允许重复元素时如何去重。

4. 排序与双指针

4.1 LeetCode 15 - 3Sum

排序后固定第一个数，在其右侧用左右指针寻找另外两个数。当前和小于 0 时左指针右移；大于 0 时右指针左移。

关键不变量：固定 first 后，答案只在 (first, left, right) 的有序搜索区间内。

去重分两层：

1. first 与前一个值相同则跳过。
2. 找到答案后，左右指针分别跨过所有相同值。

项目复制输入后排序，避免题解悄悄修改调用方数组。代价是 O(n) 额外空间。

常见错误：只对结果使用 HashSet 去重，掩盖指针逻辑问题并增加空间；三数相加直接使用 int，极端值可能溢出。

4.2 LeetCode 42 - Trapping Rain Water

位置 i 的水量由左右最高边界的较小者决定：

```text
water[i] = min(leftMax[i], rightMax[i]) - height[i]
```

预计算左右数组需要 O(n) 空间。双指针把空间降到 O(1)：当 leftMaximum <= rightMaximum 时，左侧水量已经由左最高边界确定，因为右侧至少存在不低于它的边界。

不变量：指针外侧位置已经完成计水，leftMaximum 与 rightMaximum 分别是已扫描区域最高柱。

常见错误：只比较当前左右柱，而没有维护历史最高值；把负高度当合法输入；水量累计仍使用 int 导致大输入溢出。

4.3 LeetCode 121 - Best Time to Buy and Sell Stock

卖出日扫描到 price 时，最佳买入价就是此前最小价格。维护：

```text
minimumPrice = min(此前价格)
maximumProfit = max(maximumProfit, price - minimumPrice)
```

买入必须早于卖出，因此应在同一次扫描中维护前缀最小值，不能简单使用全局最小和最大。

扩展：允许多次交易、含手续费、冷冻期时，问题会转成状态机动态规划。

4.4 LeetCode 215 - Kth Largest Element

完整排序是 O(n log n)。Quickselect 只递归包含目标下标的一侧，平均 O(n)。

将“第 k 大”转换为升序下标：

```text
targetIndex = length - k
```

Partition 完成后，pivot 已位于最终排序位置：左侧不大于它，右侧大于它。比较 pivotIndex 与 targetIndex 决定继续哪一侧。

常见错误：k 是 1 基排名而数组是 0 基；重复值导致分区死循环；直接修改输入却没有写入契约。

5. 滑动窗口与字符串

5.1 LeetCode 3 - 无重复字符最长子串

窗口 [windowStart, index] 始终没有重复字符。字典保存字符最近出现位置。发现字符在当前窗口中出现过时：

```text
windowStart = previousIndex + 1
```

必须保证 previousIndex >= windowStart，否则旧位置已经在窗口外，不应让左边界倒退。

C# 字符串按 char 遍历，即 UTF-16 code unit。若题目扩展到完整 Unicode 文本，应考虑 Rune 和组合字符。

5.2 LeetCode 76 - Minimum Window Substring

这是“满足条件时收缩”的可变滑动窗口：

1. 右边界扩张，把新字符加入计数。
2. 当所有目标字符种类都达到所需数量，移动左边界缩短窗口。
3. 移除某字符使其低于需求时，窗口重新变为不满足，继续扩张。

satisfiedKinds 统计达到目标数量的字符种类，而不是目标字符总数。目标含重复字符时，例如 AABC，A 必须出现两次才算满足。

常见错误：每移动一次都完整比较两个字典，导致额外字符集成本；只判断字符是否出现而忽略数量；收缩后忘记更新满足状态。

5.3 LeetCode 20 - Valid Parentheses

遇到左括号入栈，右括号必须与栈顶匹配。最终栈必须为空。

为什么是栈：最近打开的括号必须最先关闭，正是 LIFO。

项目使用数组和计数器实现栈，避免为单个字符再创建节点；遇到非括号字符按契约返回 false。

6. 链表题型

6.1 公共节点模型

LeetCode/Models/ListNode.cs 提供 Value、Next、FromValues 和 ToArray。FromValues 使用现代 C# 的 params ReadOnlySpan<int>，调用时仍可写：

```CSHARP
var head = ListNode.FromValues(1, 2, 3);
```

6.2 LeetCode 21 - Merge Two Sorted Lists

两个指针分别指向两个链表尚未处理的最小节点，每次把较小者接到结果尾部。

项目复用并重新连接原节点，因此额外空间 O(1)，但会改变输入链表的链接关系。生产 API 必须明确这一副作用；若要求输入不可变，应复制节点。

6.3 LeetCode 141 - Linked List Cycle

慢指针每次走一步，快指针每次走两步。有环时，两者在环内的相对距离每轮缩短一步，最终相遇。

判断必须使用节点引用身份 ReferenceEquals，不能比较节点值。不同节点完全可以保存相同值。

扩展：相遇后让一个指针回到头部，两者都每次走一步，再次相遇的位置就是环入口。

6.4 LeetCode 206 - Reverse Linked List

每轮维护：

```TEXT
previous：已经反转部分的头
current：尚未处理部分的头
next：修改 current.Next 前保存的后继
```

最常见错误是先修改 current.Next 再保存后继，导致未处理链表丢失。

7. 二叉树题型

7.1 LeetCode 102 - Level Order Traversal

队列中存放下一批待访问节点。每层开始时记录 levelSize = queue.Count，只处理这么多个节点；处理过程中加入的是下一层。

不变量：一次外层循环恰好消费一层，并把下一层完整放入队列。

常见错误：内层循环直接以不断变化的 queue.Count 为上限，导致多层混在一起。

7.2 LeetCode 236 - Lowest Common Ancestor

递归返回值表示“当前子树中找到的目标或公共祖先”：

- 当前节点就是目标时返回当前节点。
- 左右子树都返回非空，当前节点是首次汇合点。
- 只有一侧非空，把它向上传递。
- 两侧都为空，返回 null。

项目遵循题目常见前置条件：两个目标都存在于树中。若用于通用库，应同时返回“找到几个目标”，否则只有一个目标存在时会误把它当答案。

8. 图与网格

8.1 LeetCode 200 - Number of Islands

二维网格可视为隐式图：每个陆地格是顶点，上下左右陆地之间有边。扫描到未访问陆地时，岛屿数加一，并用 DFS 标记整个连通分量。

项目使用独立 visited 数组，不修改输入。常见题解把陆地改成水以节省空间，但那会产生输入副作用。

常见错误：把对角线也视为连通；访问入栈时不标记，导致同一节点重复入栈；不验证锯齿数组是否为矩形。

8.2 LeetCode 207 - Course Schedule

先修关系构成有向图。若图有环，就无法完成所有课程。

Kahn 算法：

1. 统计每门课程入度。
2. 把所有入度为 0 的课程入队。
3. 完成课程并删除其出边，使后继入度减一。
4. 最终完成数量等于课程总数，说明无环。

易错点：边方向应是 prerequisite -> course；重复先修边会重复增加入度，若允许重复输入应先去重。

9. 动态规划

9.1 LeetCode 70 - Climbing Stairs

到达第 n 阶的最后一步来自 n-1 或 n-2：

```text
ways[n] = ways[n - 1] + ways[n - 2]
```

只依赖前两项，可压缩为两个变量。项目把 0 阶定义为一种“什么也不做”的方案，并使用 checked 暴露 int 溢出。

9.2 LeetCode 198 - House Robber

处理当前房屋时只有两种互斥选择：

```text
不抢当前：oneStepBack
抢当前：twoStepsBack + currentValue
```

取两者最大值。这里的关键不是记公式，而是识别“当前选择与前一个位置冲突”。

9.3 LeetCode 139 - Word Break

reachable[end] 表示前 end 个字符可以由词典拼成。枚举最后一个单词的起点 start：

```text
reachable[start] && text[start..end] in dictionary
```

项目写法强调状态转移，但切片会创建新字符串。性能进阶可按最大单词长度限制 start，使用 Trie，或通过 Span 避免部分分配。

9.4 LeetCode 322 - Coin Change

状态是凑出金额 a 的最少硬币数：

```text
dp[a] = min(dp[a - coin] + 1)
```

每种硬币可重复使用，这是完全背包。无法到达的状态使用大于任何合法答案的哨兵值。

LeetCode API 用 -1 表示无解；项目通用算法用 int? 表示无解。适配器把 null 转成 -1，体现领域 API 与通用 API 的差异。

10. 如何独立解题

拿到新题时按以下顺序写在纸上：

1. 输入规模和数据范围。
2. 暴力解法及复杂度。
3. 是否存在有序性、单调性、重复子问题或局部状态。
4. 需要哪种结构：哈希表、栈、队列、堆、树还是图。

5 循环或递归不变量。

6 空输入、单元素、重复值、全负数、溢出和无解情况。

7 是否修改输入，返回值所有权属于谁。

模式识别速查：

题目特征	优先考虑
查找补数、频次、最近位置	哈希表
有序数组中寻找组合	双指针、二分
连续子串/子数组条件	滑动窗口、前缀和
最近打开先关闭	栈
按层、最少步数	BFS
连通块	DFS、BFS、并查集
依赖关系	拓扑排序
最优值由更小前缀决定	动态规划
第 k 大、不要求完整排序	堆、Quickselect
枚举所有组合且可提前排除	回溯

11. 测试策略

每题至少覆盖：

- 官方风格示例。
- 空输入或最小合法输入。
- 重复元素。
- 无解输入。
- 极端数值和整数溢出。
- 不应修改输入时的输入快照。
- 链表和树使用节点身份，而不是只比较值。

可以进一步加入确定性随机差分测试：

- Two Sum 与 $O(n^2)$ 暴力解比较。
- 3Sum 与三重循环 + 集合规范化比较。
- Quickselect 与完整排序比较。
- Minimum Window 与枚举全部子串比较。
- Number of Islands 与并查集实现比较。

12. 四周练习计划

第 1 周：数组与字符串

按顺序完成 1、121、3、20、15、76。重点掌握哈希表、固定/可变窗口和双指针去重。

第 2 周：链表与树

完成 21、206、141、102、236。每题先画指针或递归返回值，不直接写代码。

第 3 周：图

完成 200、207，再回到项目中的 Dijkstra、Bellman-Ford 和 Kruskal，比较网格图、依赖图和带权图。

第 4 周：动态规划与选择

完成 70、198、139、322、215。每道 DP 题必须写状态定义、转移、初始化和答案位置。

13. 验收标准

完成专题后，应能够：

- 不看代码写出 Two Sum、无重复窗口和有效括号。
- 正确处理 3Sum 去重和雨水双指针不变量。
- 原地反转链表并解释为什么不会丢失后继。
- 使用快慢指针按节点身份检测环。
- 使用队列按层遍历树并递归求 LCA。
- 把岛屿问题建模为连通分量，把课程问题建模为拓扑排序。
- 为爬楼梯、打家劫舍、单词拆分和零钱兑换定义 DP 状态。
- 解释 Quickselect 为什么平均 O(n)，以及它的最坏情况。
- 在 C# 中正确处理 nullable 节点、集合表达式、Span、比较器和整数溢出。
- 为新题先写不变量与边界测试，再写实现。

刷题的最终目标不是提高已完成题目数量，而是面对新题时能从约束、结构和不变量推导出解法。

第六篇 P3 数论、位运算与高级算法

> 补齐数论与位运算，形成高级图、字符串和动态规划算法族，并讲解质量门禁、结构化追踪和 14 道非重复专题题。

title: P3 数论、位运算与高级算法学习指导 description: 用 14 道专题题学习数论、位运算、网络流、高级字符串与高级动态规划。

1. P3 为什么不继续机械增加题量

原有仓库已经覆盖常见线性结构、树、图、字符串、动态规划和 100 道 LeetCode 题。P3 的目标不是把数字从 100 改成 150，而是补齐三种此前不足的能力：

1. **整数推理能力**：通过数论和位运算学习溢出、模运算、二进制状态与输出规模。
2. **算法族比较能力**：在已有 Floyd、Dinic、后缀数组和普通 DP 旁边加入 Johnson、Hopcroft-Karp、后缀自动机及三类高级 DP。
3. **证据与观察能力**：使用差分测试、覆盖率阈值、结构化 Trace 和 Mermaid，而不是只依赖几个样例。

每个 P3 模块仍遵循同一个学习顺序：先写输入契约，再找不变量，随后推导复杂度，最后用示例、性质或差分测试验证。

2. P3 知识地图

专题	主要实现	必须理解的不变量	典型复杂度
数论	gcd、lcm、扩展欧几里得、模快速幂、模逆元、筛法、质因数分解、矩阵快速幂	gcd 不因取余改变；模逆元要求互质	O(log n) 到 O(n log log n)
位运算	置位计数、2 的幂、汉明距离、位反转、子掩码和子集枚举	每次删除一个最低置位；子集输出本身为指数规模	O(popcount) 到 O(2^n)
高级图	Johnson、Hopcroft-Karp、欧拉路径、倍增 LCA、最小割恢复	重标边非负；匹配端点唯一；每条欧拉边只使用一次	依算法从 O(V+E) 到全源最短路
高级字符串	Manacher、后缀自动机、Rune 滑动窗口	镜像半径不越过已知右边界；suffix link 指向最长真后缀状态	O(n)
高级 DP	区间 DP、树形 DP、状态压缩 DP	区间最后决策拆开子问题；树节点保留选/不选状态	O(n) 到 O(n²2^n)
专题题库	6 道数值题 + 8 道高级题	题目实现复用上述结构而不是复制模板	依题目而定
工程证据	格式门禁、编译警告门禁、覆盖率阈值、变异测试	同一命令在本地与 CI 得出一致结果	持续集成成本

3. 数论：为什么欧几里得算法有效

实现位置：DataStructureAndAlgorithm/NumberTheory/NumberTheoryAlgorithms.cs。

3.1 gcd 为什么可以不断取余

设 a = qb + r。任何同时整除 a 和 b 的数也会整除 r = a - qb；反过来，同时整除 b 和 r 的数也会整除 a。因此：

```text
gcd(a, b) = gcd(b, a mod b)
```

每次非零余数都严格小于除数，所以算法一定终止。代码使用 ulong 返回绝对值，因为 Math.Abs(long.MinValue) 无法放进 long。先计算 -(value + 1) 再加一，可以安全得到 2^63。

3.2 lcm 为什么先除后乘

数学关系是：

```text
lcm(a, b) = |a| * |b| / gcd(a, b)
```

代码写成 |a| / gcd * |b|，结果相同，但中间值更小。最终结果仍可能超过 ulong，因此使用 checked 明确抛出溢出异常，而不是静默回绕。

3.3 扩展欧几里得为什么返回 BigInteger

扩展算法除了 gcd，还维护：

```text
a * x + b * y = gcd(a, b)
```

回代系数可能在中间步骤超出 long。P3 返回 BezoutIdentity，并用 BigInteger 保存 gcd 和系数，使 API 对整个 long 输入域都成立。测试不只检查某个预期系数，而是重新计算等式；因为贝祖系数不唯一，验证关系比验证固定答案更正确。

3.4 模快速幂为什么是对数复杂度

指数的二进制展开决定哪些平方项要乘入结果。例如 13 = 8 + 4 + 1，所以只需要连续平方并选取对应项。每轮把指数右移一位，循环次数是 O(log exponent)。

乘法在取模之前可能溢出 long，因此教学实现使用 BigInteger 保存中间乘积。这里的重点是理解二进制快速幂；生产中的固定宽度高性能实现还可以继续学习无溢出乘法或 Montgomery Reduction。

3.5 模逆元为什么必须互质

value 的模逆元是满足 value * inverse ≡ 1 (mod modulus) 的数。根据贝祖等式，只有 gcd(value, modulus) = 1 时，右侧才能组合出 1。代码先运行扩展欧几里得，不互质就抛出 ArgumentException，避免返回一个看似正常但数学上不存在的结果。

3.6 筛法为什么从 p^2 开始

处理质数 p 时，2p、3p ... (p-1)p 都包含比 p 更小的质因子，已经被之前的轮次标记。因而从 p^2 开始既正确又减少重复工作。循环条件写成 p <= n / p，避免计算 p * p 时发生整数溢出。

3.7 Fibonacci 为什么使用矩阵快速幂

Fibonacci 递推可以表示为固定矩阵的幂。矩阵乘法满足结合律，所以能够使用与标量快速幂相同的二进制拆分。复杂度从线性下降为 O(log n)，并用 BigInteger 展示第 100 项以后仍然正确的结果。

4. 位运算：什么时候位技巧是真正的状态表达

实现位置：DataStructureAndAlgorithm/BitManipulation/BitAlgorithms.cs。

4.1 value & (value - 1) 为什么删除最低置位

减一会把最低的 1 变成 0，并把它右边的零全部变成一；再与原值按位与，右侧新增的一被清掉，最低置位也被删除。因此置位计数的循环次数等于一的数量。

业务代码通常应使用 BitOperations.PopCount，因为运行时可以利用 CPU 指令。仓库同时保留教学实现和框架实现，并用测试做结果对照。

4.2 为什么零不是 2 的幂

value & (value - 1) == 0 对零也成立，所以必须先判断 value != 0。位公式如果缺少定义域，常常会在边界上产生貌似合理的错误。

4.3 子掩码枚举为什么使用 (submask - 1) & mask

减一把当前子掩码跳到更小的二进制组合，再与原掩码相交会删除所有不属于原集合的位。序列最终到零，且不会重复。

4.4 为什么子集枚举限制 30 个元素

算法可以写得很短，但输出数量是 2^n。限制不仅防止 1 << n 的位移回绕，也提醒学习者：惰性枚举只能推迟成本，不能消除指数级输出。

5. Johnson：稀疏图为什么不总用 Floyd

实现位置：DataStructureAndAlgorithm/Graph/P3GraphAlgorithms.cs。

Floyd-Warshall 的优势是实现稳定、支持负边且查询方便，但时间固定为 $O(V^3)$。Johnson 针对稀疏图：

1. 建立一个到所有顶点权重为零的假想超级源点。
2. 用 Bellman-Ford 求每个顶点的势能 h(v)。
3. 把边权改写为 w'(u,v) = w(u,v) + h(u) - h(v)。
4. 在非负的新边权上从每个顶点运行 Dijkstra。
5. 用势能还原原始距离。

任意路径的势能项会首尾抵消，所以同一起点终点之间的路径相对大小不变。Bellman-Ford 又保证新边权非负；若仍能松弛，则原图含负环，最短路没有定义。

测试同时运行 Johnson 和 Floyd-Warshall，逐顶点对比较距离。这种差分测试比只检查一条路径更有证据力。

6. Hopcroft-Karp：为什么一次寻找多条增广路

普通二分图匹配每找到一条增广路就修改匹配，最坏需要反复扫描大量相同边。Hopcroft-Karp 先用 BFS 建立最短增广路层次，再用 DFS 在同一层次图中找一组互不冲突的增广路。

匹配始终维护两个方向的映射：left -> right 与 right -> left。任何更新都必须同时修改两边，否则下一轮会把已经匹配的端点再次使用。

7. 欧拉路径：为什么答案要逆序构造

Hierholzer 不在第一次走边时立刻把顶点写进最终答案，因为当前选择之后可能还要插入一个环。算法一直走到无边可走，才把当前顶点加入逆序答案；回退过程中，局部环自然插入正确位置。

有向图欧拉路径的度数条件是：

- 起点 out - in = 1；
- 终点 in - out = 1；
- 其他顶点入度等于出度；
- 或者所有顶点入度等于出度，形成欧拉回路。

度数条件还不够，最终路径长度必须为 edgeCount + 1。否则说明存在与起点不连通的边分量。

8. 倍增 LCA：为什么预处理祖先的 2 的幂

BinaryLiftingTree<TVertex> 保存每个节点的 2^k 级祖先。查询时把深度差按二进制拆开，就能在 O(log n) 内把较深节点提升到同一深度；随后从最大的幂向下尝试同时提升两个节点，最后它们的父节点就是 LCA。

构造函数要求边数等于顶点数减一，并验证从根能够访问全部顶点。这样后续查询不必每次重新怀疑输入是否真的是树。

9. 最大流与最小割为什么要一起学习

实现位置：DataStructureAndAlgorithm/Graph/P3FlowAlgorithms.cs。

P3 的追踪版 Dinic 返回：

- 最大流数值；
- 每条原始边的容量与实际流量；
- 最小割的源侧顶点集合。

最大流结束时，残量网络中已经不存在从源到汇的路径。从源点沿正残量边仍可到达的顶点构成集合 S，其余顶点构成 T。原网络中所有 S -> T 边都已经饱和，容量和等于最大流。

测试会从结果中重新计算割容量并与最大流比较，实际验证 max-flow = min-cut，而不是只相信实现返回的数值。

10. Manacher：为什么镜像只能提供下界

实现位置：DataStructureAndAlgorithm/PatternMatching/P3StringAlgorithms.cs。

Manacher 维护当前最靠右的回文区间。新中心位于该区间内时，它关于区间中心的镜像已经计算过，因此可以复用镜像半径；但复用范围不能超过当前右边界，因为边界以外还没有比较证据。

代码分别计算奇数长度与偶数长度回文，避免在字符串中插入特殊分隔符，也避免分隔符与真实输入字符冲突。

11. 后缀自动机：状态为什么能代表多个子串

后缀自动机的一个状态代表一组结束位置集合相同的子串。MaximumLength 是其中最长长度，suffix link 指向最长的、结束位置集合更大的真后缀状态。

新状态贡献的不同子串数量为：

```text
state.MaximumLength - link(state).MaximumLength
```

构造时出现转移长度不连续，就复制目标状态形成 clone。clone 继承转移和 suffix link，但缩短最大长度，使自动机仍保持确定性和最小状态结构。

P3 支持：

- 判断模式是否为子串；
- 统计不同子串数量；
- 在线计算与另一个字符串的最长公共子串长度。

12. Rune：为什么 char 不等于一个用户可见字符

C# 的 char 是 UTF-16 code unit。非 BMP 字符（例如许多 emoji）由一对代理项组成，按 char 滑动可能把一个 Unicode 标量拆开。

LongestSubstringWithoutRepeatedRunes 使用 string.EnumerateRunes()，按 Rune 去重，同时记录 UTF-16 起点和长度。这样结果既按标量值正确计数，也可以安全传给 Substring。

需要注意，Rune 仍不等于完整的“用户可见字素簇”；组合音标和 emoji 序列还需要更高层的文本分割规则。

13. 三类高级动态规划为什么要分开学习

实现位置：DataStructureAndAlgorithm/DynamicProgramming/P3DynamicProgrammingAlgorithms.cs。

13.1 区间 DP

矩阵链和 Burst Balloons 都先确定区间边界，再枚举最后一次合并或最后一个决策。关键不是二维数组，而是“选定最后决策后，左右子问题相互独立”。

13.2 树形 DP

树上最大权独立集为每个节点维护：

- Take：选择当前节点，所有孩子必须 Skip；
- Skip：不选当前节点，每个孩子可取 Take/Skip 较大值。

实现使用显式栈建立父子顺序，再逆序计算状态，避免深树递归造成调用栈溢出。

13.3 状态压缩 DP

Held-Karp TSP 的状态 (mask, last) 表示已访问 mask 中的顶点且当前停在 last。位掩码让集合成为数组下标，但状态数量仍是 n * 2^n，所以 API 明确限制最多 18 个顶点。

状态压缩不是把指数问题变成多项式，而是用可控内存消除重复搜索。

14. 新增 14 道不重复专题题

原有 100 题编号保持不变，P3 题库独立追加以下题号，集合之间没有重复。

题号	方法	为什么这样做	核心不变量	复杂度
50	二进制快速幂	指数每轮减半	result 乘已选幂，value 保存下一平方项	`O(log
69	整数二分	避免浮点误差	answer 始终是不超过平方根的可行值	O(log n)
191	Kernighan 置位计数	每轮删除一个 1	循环次数等于剩余置位数	O(popcount)
204	埃氏筛	批量复用合数标记	未标记且不小于 2 的数为质数	O(n log log n)
231	单置位判断	2 的幂只有一个 1	零必须单独排除	O(1)
338	低位状态复用	i & (i-1) 比 i 小	当前值等于前驱置位数加一	O(n)
312	区间 DP	枚举最后戳破气球	最后决策时区间边界固定	$O(n^3)$
332	Hierholzer + 小根堆	所有票是有向多重边	逆序结果中的边都已走尽	O(E log E)
337	树形 DP	相邻节点不能同时选	每节点同时返回 Take/Skip	O(n)
516	一维区间 DP	左右字符决定是否配对	previousDiagonal 保存更新前左上状态	$O(n^2)$ / O(n)
698	降序回溯	大数先放更早暴露失败	桶和不超过目标；等值空桶等价	指数级
847	多源状态 BFS	起点未知，状态还要包含访问集合	第一次到达状态即最短	O(n2^n)
1044	后缀数组最大 LCP	重复串来自相邻有序后缀	LCP 表示相邻后缀公共前缀	$O(n \log^2 n)$
1192	Tarjan low-link	子树无法绕回才是桥	low 保存可到达的最早发现时间	O(V+E)

实现集中在：

- DataStructureAndAlgorithm/LeetCode/NumericAndBitProblems.cs
- DataStructureAndAlgorithm/LeetCode/AdvancedTopicProblems.cs
- DataStructureAndAlgorithm.Test/LeetCodeTest/P3TopicProblemsTest.cs

15. 结构化 Trace 和 Mermaid 为什么分层

IAlgorithmTraceSink 仍是核心算法唯一依赖。算法不知道结果会写入控制台、JSON、日志还是 UI。P3 新增：

- 欧几里得每轮余数追踪；
- 矩阵链每次更优分割追踪；
- Dinic 层次图、增广流和最小割追踪；
- MermaidTraceRenderer，把事件序列转成 Markdown 可嵌入的流程图。

运行 JSON：

```powershell
dotnet run --project DataStructureAndAlgorithm.P3Demo -- all json
```

运行 Mermaid：

```powershell
dotnet run --project DataStructureAndAlgorithm.P3Demo -- dinic mermaid
```

事件中的 State 必须是快照。若直接保存调用者的可变 Dictionary，后续修改会篡改历史步骤，演示结果就不再可信。

16. CI 质量门禁为什么这样设置

P3 新增以下仓库级配置：

- Directory.Build.props：统一 C# 15、nullable、确定性构建和编译器警告策略；
- Directory.Build.rsp：固定 .NET 11 默认分析模式，避免一次启用全部风格规则产生数百条历史噪声；
- .editorconfig：统一字符集、换行、缩进和现代语法建议；
- .github/workflows/quality.yml：格式、Release 构建、测试和覆盖率阈值；
- tools/check_coverage.ps1：解析 Cobertura 总覆盖率并低于阈值时失败。

阈值设置为行覆盖率 80%、分支覆盖率 70%。P3 完成时实测基线明显高于阈值，保留余量是因为覆盖率只能说明代码是否执行，不能单独证明断言质量。关键结构继续使用 Stryker 变异测试验证“测试能否杀死错误”。

格式门禁使用：

POWERSHELL

```
dotnet format DataStructureAndAlgorithm.slnx --verify-no-changes --no-restore
```

Windows 本机若禁止 .ps1，可只对仓库脚本使用一次性策略，不要修改全局系统策略：

POWERSHELL

```
powershell -NoProfile -ExecutionPolicy Bypass -File tools/check_coverage.ps1 `
  -Report path/to/coverage.cobertura.xml -MinimumLineCoverage 80 -MinimumBranchCoverage 70
```

17. 推荐八周学习顺序

1. 第 1 周：手算 gcd、扩展 gcd、模快速幂；用 BigInteger 做差分测试。
2. 第 2 周：筛法、质因数分解、矩阵快速幂和 6 道数值题。
3. 第 3 周：子掩码、子集枚举、状态压缩 TSP 和 847。
4. 第 4 周：Johnson 与 Floyd 差分，Hopcroft-Karp 和欧拉路径。
5. 第 5 周：倍增 LCA、Dinic 最小割恢复和 1192。
6. 第 6 周：Manacher、后缀自动机、Rune 和 1044。
7. 第 7 周：区间 DP、树形 DP，完成 312、337、516、698。
8. 第 8 周：输出 JSON/Mermaid Trace，阅读覆盖率与变异报告，独立重写一个模块。

18. P3 验收清单

完成 P3 后，应能够：

- 证明欧几里得取余为什么不改变 gcd，并处理 long.MinValue。
- 说明模逆元存在的充要条件，而不是只背公式。
- 区分位运算减少常数、状态压缩消除重复与指数输出无法消除三件事。
- 根据图稠密度在 Floyd 和 Johnson 之间选择。
- 解释 Hopcroft-Karp 为什么按层批量寻找增广路。
- 验证一条欧拉路径确实使用每条边一次。

- 从最大流残量网络恢复最小割并验证容量相等。
- 解释 Manacher 的镜像半径为什么不能越过当前右边界。
- 解释后缀自动机 clone 的必要性和不同子串计数公式。
- 在 Unicode 输入中区分 char、Rune 和用户可见字素。
- 分别识别区间 DP、树形 DP 与状态压缩 DP 的状态定义。
- 运行 P3 Demo 并把结构化事件渲染为 Mermaid。
- 解释覆盖率、性质测试、差分测试与变异测试分别能提供什么证据。

真正完成 P3 的标志，是能够为新问题选择证据和算法族，而不是仅仅记住新增方法名。

第七篇 数据结构与算法综合项目实战

> 通过城市配送、迷你全文检索、项目资源排程和迷你存储引擎，把自实现结构组合成端到端应用，并用版本化追踪、模型测试、跨平台 CI 与基准趋势持续证明跨模块不变量。

title: 数据结构与算法综合项目实战学习指导 description: 通过配送、搜索、调度与存储四个场景，把算法选择、不变量、追踪和测试证据连成系统。 search: false

1. 为什么需要综合项目

单独实现堆、哈希表、图或动态规划，训练的是局部不变量；真实需求却通常是“让更多订单按时送达”“让搜索结果更相关”“在资源有限时缩短工期”或“让数据在进程重启后仍可恢复”。综合项目要完成的关键转换是：

1. 把业务语言翻译为顶点、边、键、区间、状态、约束和目标函数。
2. 先写清楚优化目标的优先级，再选择算法，避免“算法能跑，但优化的是另一件事”。
3. 让每种数据结构只承担它擅长的职责，不把所有状态都塞进一个字典。
4. 把算法结果恢复为路线、搜索命中、任务时间表或存储记录。
5. 明确模型没有表达什么，并用不变量、模型测试、性质测试和差分测试持续提供证据。

本阶段提供四个可运行的 .NET 11 / C# 15 控制台项目：

项目	应用场景	主要数据结构	主要算法
CityDelivery V2	城市即时配送与车辆分单	开放寻址哈希表、带权图、并查集、LRU、二叉堆	A*、最小费用最大流（生产路径）、穷举匹配 oracle（测试对照）、Prim
MiniSearch V2	小型文档搜索、补全与纠错	位置倒排表、Trie、BK-tree、LRU、二叉堆	BM25、短语/邻近检索、Aho-Corasick、编辑距离
ProjectScheduling V2	有依赖和资源上限的项目排期	DAG、二叉堆、懒标记线段树	Kahn、关键路径 DP、贪心列表调度、小规模精确搜索
MiniStorage	可恢复的教学型键值存储	B+ 树、Bloom Filter、LFU、WAL	点查、范围扫描、预写日志、崩溃恢复、墓碑删除

“尽量自己实现”不等于拒绝 BCL。核心索引、缓存、图算法、区间结构和恢复流程优先使用仓库中的教学实现；JSON 序列化、字符串构建和只读结果集合继续使用标准库。判断标准是：如果替换这个组件会改变本项目要学习的不变量或复杂度，它就值得手写；如果它只是外围基础设施，应优先使用成熟库。

2. 四个项目共用的分层原则

每个项目都保留三层职责：

- 领域记录表达输入和输出，不在属性访问器里偷偷执行算法。
- 规划器或引擎把领域约束翻译成算法输入，并把内部结果恢复为领域结果。
- Program.cs 只构造可重复样例，并选择普通 JSON、追踪 JSON 或 Mermaid 输出。

这样写不是为了形式上的“分层”，而是为了建立可验证的依赖方向：算法层不知道控制台和序列化格式；测试可以直接调用引擎；同一套核心逻辑也可以接到 Web、桌面或文件输入。代码中仍允许 List<T> 和数组承担连续缓冲区职责，不应为了“全手写”而制造没有教学价值的包装。

另一个共通原则是先定义契约：空输入怎样处理、重复键是否覆盖、区间是左闭右开还是双闭、相同得分如何稳定排序、缓存键包含哪些版本状态。综合项目中的许多错误并不来自算法公式，而来自两个模块对同一契约理解不同。

3. 项目一：城市即时配送 V2

3.1 应用场景与两层目标

配送站接收订单和空闲车辆。车辆必须满足载重、方向可达和截止时间约束，每辆车在一轮中最多领取一个订单。V2 把目标明确为字典序优化：

1. 首先最大化成功分配数量；
2. 在最大分配数量相同的方案中，最小化总旅行时间。

为什么必须分两层写？如果只最小化费用，算法可能选择“不派任何车”，费用为零但没有业务价值；如果只做最大匹配，同样能配送三单的两个方案中可能选到绕远方案。目标函数必须先于算法选择。

3.2 为什么路线图和维护图分开

旅行时间图是有向图，因为单行道只允许一个方向；维护网络回答设施之间是否要保留连接，通常按无向图处理。项目因此从同一批道路构造两个视图：

- `WeightedGraph<string>(isDirected: true)` 保存旅行分钟数；
- `WeightedGraph<string>(isDirected: false)` 保存维护成本。

把二者混成一张图，会把“能否按方向行驶”和“物理设施是否连接”混为一谈。Prim 运行在无向维护图上，A* 运行在有向旅行图上，这是领域语义决定的数据结构边界。

3.3 为什么先并查集预筛，再运行 A*

并查集近似常数时间判断两地是否处于同一弱连通分量。若不连通，候选必然不可达；若弱连通，仍不代表存在有向路径，所以还要运行 A*。这是“便宜的必要条件 + 较贵的充分验证”：

1. 容量不足、弱连通失败时立即拒绝；
2. 其余候选用 A* 验证方向和最短旅行时间；
3. 再检查截止时间，得到可行的车辆—订单边。

输入没有能证明可采纳性的地理启发函数，因此项目使用 `h(n) = 0`。A* 此时退化为 Dijkstra，但仍保持最优性。宁可选择可证明正确的退化形式，也不应使用可能高估剩余成本的“看起来更聪明”的启发函数。

3.4 从最大匹配升级到最小费用最大流

Hopcroft-Karp 只回答“最多能匹配多少对”，不优化匹配边的费用。V2 的生产路径直接构造下面的流网络； 测试再用小规模穷举最大匹配作为独立参照，避免“流算法自己证明自己”：

```text
源点 -> 每辆车 -> 每个订单 -> 汇点
```

- 源点到车辆、订单到汇点的容量均为 1，保证一车一单。
- 车辆到订单只为可行候选建边，容量为 1，费用为最短旅行分钟数。
- 每轮用 Johnson 势能把残量边的约化费用保持为非负，再用项目自实现二叉堆运行 Dijkstra。
- 一直增广到残量网络不存在源汇路径；此时流量自然达到最大值。配送网络的源边容量为 1，

所以每轮最多增加一个分配，而通用实现仍允许一次发送整条路径的瓶颈流量。

这样写把“流量”和“费用”分开：流量证明第一目标，费用优化第二目标。测试也应分别断言分配数等于最大匹配、总旅行分钟数等于小规模穷举 oracle，而不能只看某一组具体匹配编号。

残量网络中的反向边费用取相反数，它允许后续增广撤销早先的局部选择。若没有反向边，算法会退化为不可回退的贪心选择，不能保证全局最小费用。

3.5 缓存、确定性与复杂度

路线缓存键包含 (MapVersion, From, To)。道路变化后即使起终点相同，旧路线也不再属于同一查询。输出总是显式排序，不依赖哈希表枚举顺序；费用相同时使用稳定的车辆和订单编号打破平局，使测试、追踪和不同平台输出可复现。

设地点数为 V、开放道路数为 E、车辆数为 C、订单数为 O、可行边数为 M、最终分配数为 F：

- 单次零启发 A*：O((V + E) log V)；
- 最坏候选构造：O(CO(V + E) log V)；
- 小规模测试 oracle 可用 Hopcroft-Karp 或完整枚举核对最大基数；它不在 V2 派单生产路径上；
- 带势函数的逐次最短增广路约为 O(F · (C + O + M) log(C + O))；
- Prim 最小生成森林：O((V + E) log V)。

限制也要明确：教学版没有地理空间索引、批量最短路、动态交通权重和并发订单锁；最小费用流虽然实现了 Johnson 势能， 但使用 double 成本，也没有负原始费用初始化、cost-scaling 或大规模内存调优。 约化成本的理论值应非负，但大势能加减会吞掉低位；实现按操作数尺度计算 8 ULP 误差预算，只把预算内的小负数钳为零，Dijkstra 松弛仍严格比较每个可表示成本，避免 epsilon 把真实便宜路径当平局。

3.6 运行与阅读顺序

POWERSHELL

```
dotnet run --project DataStructureAndAlgorithm.Scenarios.CityDelivery -c Release
dotnet run --project DataStructureAndAlgorithm.Scenarios.CityDelivery -c Release -- --trace
json
dotnet run --project DataStructureAndAlgorithm.Scenarios.CityDelivery -c Release -- --trace
mermaid
```

阅读时先手算两车两单：列出可行边、最大匹配数，再枚举同流量方案比较总费用。随后对照追踪中的候选评估、最大匹配、费用优化和派单阶段。这样能看出 V2 不是“换一个更复杂算法”，而是让算法与业务目标完全对齐。

4. 项目二：迷你搜索引擎 V2

4.1 为什么第一版词频索引不够

第一版倒排表记录“词出现在哪些文档、出现多少次”，足以完成 AND 查询和简单词频排序，却回答不了：

- 两个词是否连续出现；
- 两个词相距是否不超过指定窗口；
- 常见词和稀有词是否应有相同权重；
- 大词典纠错是否必须扫描每个词。

V2 因此把 posting 扩展为“文档编号 + 词频 + 有序位置列表”。位置索引多占用空间，却把短语和邻近检索从重新扫描全文变成 posting 内的位置检查。这是典型的以索引空间换查询时间。

4.2 位置列表为什么保持有序

分词时按文本顺序追加位置，天然得到递增列表。查询短语 data structure 时，要寻找 position(structure) = position(data) + 1；邻近查询则检查位置差是否落在允许窗口内。

当前教学代码为了让位置判定最直观，使用顺序查找和位置对枚举，只检查 posting 而不重新扫描正文； 下一步可在不改变公开语义的前提下升级为双指针，把两个位置表的合并降为线性时间。 有序仍是这个优化的前提。若位置单位在“字符偏移”和“词元序号”之间混用，短语语义也会错误； 代码选择词元序号，是因为它不受单词字符长度影响，且邻近窗口更容易解释。

4.3 为什么使用 BM25，而不是直接累加词频

直接累加词频会偏爱长文档，也会让到处出现的常见词占据过高权重。BM25 同时考虑：

- TF：词在当前文档中的出现次数，但收益逐渐饱和；
- IDF：词越稀有，区分能力越强；
- 文档长度归一化：减少长文档仅因词多而获益的偏差。

教学实现把公式拆成命名清晰的中间量，而不是压成一行。这样测试可以分别验证文档频率、平均文档长度和最终排序。词频饱和项把分子、分母同时按 max(1, k1) 缩放，使 k1 = double.MaxValue 时仍得到有限的等价结果。 浮点公式测试使用容差，公开排序则严格比较实际 double；只有分数完全相等才按文档编号决胜，避免 epsilon 比较器破坏传递性。

BM25 仍不是“相关性真理”：它不了解同义词、语义和用户意图。项目的目的，是展示目标函数如何从朴素频次升级为可解释的统计权重。

4.4 短语、邻近和高亮分别解决什么

- 短语检索要求词序和相邻关系完全匹配。
- 邻近检索允许两个词之间有有限间隔，适合表达“概念彼此靠近”。
- Aho-Corasick 在最终少量文档中一次扫描恢复多个查询词的字符命中位置，用于展示高亮。

位置倒排表负责缩小候选和判定词元关系，Aho-Corasick 负责最终文本展示。二者看似都处理“位置”，实际处于不同流水线阶段，不应合并为一个职责模糊的结构。 分词与整词高亮共享同一套 Rune 规则；高亮虽然返回 UTF-16 下标，判断左右边界时仍解码完整 Unicode 标量，不能把辅助平面字母的高、低代理项误当分隔符。

4.5 为什么纠错升级为 BK-tree

朴素纠错对词典中每个词计算 Levenshtein 距离，规模增大后成本明显。BK-tree 利用编辑距离满足三角不等式：若当前节点与查询词距离为 d，允许误差为 r，只需访问边标签位于 [d-r, d+r] 的子树。

代码仍保留编辑距离作为度量，BK-tree 只负责剪枝搜索空间。插入时，以“新词到当前词的距离”选择子边；相同距离继续下沉。这个结构的性能依赖词典分布，最坏仍可能接近线性扫描，因此文档和基准不把它宣传为无条件 O(log n)。

4.6 Top-K、缓存与复杂度

兼容的 V1 AND 查询在候选很多而只需要前 K 个时维护容量为 K 的最小堆，复杂度为 O(N log K)。 V2 BM25 为了让实现保持简单、确定且便于和独立公式逐项核对，当前先计算并稳定归并排序全部候选，再截取前 maxResults 个结果，复杂度为 O(N log N)；它严格按实际返回的 double 分数降序，只有分数完全相等时才以文档编号决胜。 独立公式测试用容差核对数值，但排序比较器本身不能使用会破坏传递性的 epsilon。生产级 Top-K 可以改用容量为 K 的堆，把排序阶段降为 O(N log K)。

V1 兼容查询的缓存键包含索引版本、规范化查询和结果数，新增文档后版本变化，旧缓存自然失效。 V2 的 BM25、短语和邻近入口目前不缓存，避免在教学第一版里遗漏 k1、b、模式或距离参数而错误复用结果； 若以后增加高级查询缓存，所有这些参数都必须成为缓存身份的一部分。

设总分词量为 T、posting 访问量为 P、候选数为 N、最终结果数为 K：

- 建立位置索引平均 O(T)；
- AND/BM25 候选累计约 O(P)；
- 当前短语/邻近位置判定最坏为位置数乘积；双指针版本可降为两个位置表长度之和；
- V1 Top-K 为 O(N log K)，V2 BM25 完整排序为 O(N log N)；
- BK-tree 平均访问节点数取决于词典分布，最坏 O(vocabularySize · editDistanceCost)。

教学版限制包括：按 Rune 识别字母数字但没有语言相关分词、Unicode normalization、增量合并和删除段；BM25 提供可配置 k1/b，但没有训练或语言相关调参。

4.7 运行与阅读顺序

POWERSHELL

```
dotnet run --project DataStructureAndAlgorithm.Scenarios.MiniSearch -c Release
dotnet run --project DataStructureAndAlgorithm.Scenarios.MiniSearch -c Release -- --trace
json
dotnet run --project DataStructureAndAlgorithm.Scenarios.MiniSearch -c Release -- --trace
mermaid
```

建议先画出一个三文档位置倒排表，再分别运行普通、BM25、短语和邻近查询。最后故意输入拼写错误，观察 BK-tree 如何缩小编辑距离候选。不要一开始只看最终排名；先验证 posting 与位置不变量，排名错误才有可定位的上游原因。

5. 项目三：项目调度 V2

5.1 两个问题：得到可行解与证明差距

任务有持续时间、资源需求和前置依赖，团队资源容量有限。系统必须拒绝未知依赖、自依赖和环，并输出满足依赖与容量约束的计划。V1 的关键路径优先列表调度能快速得到可行解，但启发式不保证最短工期。

V2 同时提供两条路径：

- 贪心调度处理常规规模，快速得到确定性可行解；
- 小规模精确搜索枚举并剪枝，得到最优工期，作为教学 oracle。

精确解的价值不仅是“再给一个答案”，更是量化贪心解与最优解之间的差距。没有 oracle 时，一个看起来合理的 makespan 很难说明质量如何。

5.2 DAG、拓扑序和关键路径

若 B 依赖 A，图中添加 A -> B。Kahn 算法按入度归零推进；零入度集合使用最小堆，确保多个合法拓扑序中优先选择较小编号，输出可复现。若处理顶点数少于任务数，剩余部分必有环。

关键剩余长度按反向拓扑 DP 计算：

TEXT

```
remaining(task) = duration(task) + max(remaining(dependent))
```

ready queue 优先考虑 remaining 较大的任务，减少阻塞长链的风险。它是启发式优先级，不是最优性证明；文档和代码注释都应明确这一边界。

5.3 为什么升级为“区间加 + 区间最大值”懒标记树

时间离散为左闭右开槽位 [start, end)。判断任务能否放入某窗口，只需查询窗口最大占用并检查：

```text
QueryMax(start, end) + resourceUnits <= capacity
```

任务落位后，整个区间占用增加 resourceUnits。V1 逐点更新需要 O(duration · log H)；V2 的懒标记线段树把区间增加推迟到真正访问子节点时，一次预留降为 O(log H)。

懒标记的关键不变量是：节点最大值和最小值都已包含尚未下推的统一增量；访问子节点前必须 PushDown，合并父节点时分别取子节点最大/最小。最大值证明正增量安全，最小值证明负增量不会下溢；内部用 Int128 保存延迟增量，因为两个合法 long 值之差可能需要 65 位。 更新会先预计算并记录触及节点快照，任何溢出都整体回滚；因此一次 RangeAdd 成功后，未来查询不会在下推时才发现潜伏溢出。区间统一采用 [start, end)，持续时间自然满足 end - start = duration，相邻任务不会重复占用边界槽位。

5.4 小规模精确解为什么必须设置上限

资源约束项目调度是组合爆炸问题。精确求解器先固定确定性拓扑顺序，再为每个任务枚举所有可能改进上界的整数起点，使用当前最好工期做上界，并用依赖链、剩余工作量等下界剪枝。即使如此，任务数和时间跨度增长后仍会指数膨胀。

因此公开 API 明确限制为最多 8 个任务、持续时间总和最多 32。内部另有 2,000,000 个搜索节点预算： 即使输入满足前两项，病态实例在预算内仍未穷尽时也会抛出 InvalidOperationException；启用追踪时，抛出前还会记录 ExactSearchAborted。 算法不会把当前最好计划冒充为已证明最优。只有正常返回的比较结果才携带最优性证明。测试通过 internal 构造参数把预算降为 1，在第二个节点稳定触发中止，从而不必真的消耗两百万节点也能持续验证该契约。 生产系统通常会采用 CP-SAT、整数规划或专业调度器。

成功完成的精确搜索还必须保持确定性分支顺序；否则不同运行虽都得到最优解，搜索轨迹和同工期方案可能变化，给教学追踪和回归测试带来噪声。

5.5 应验证哪些不变量

每个计划都必须满足：

1. 每个任务恰好出现一次；
2. Start >= 0 且 End = Start + Duration；
3. 开始时间不早于全部依赖的结束时间；
4. 任意时间槽的总资源不超过容量；
5. 拓扑序中每条依赖边起点都在终点之前；
6. 精确求解器返回的工期不大于同输入的贪心工期；
7. 在足够小的输入上，精确结果与独立穷举 oracle 一致。

只断言 makespan 不够：遗漏任务也可能产生很小的工期。先验证可行性不变量，再比较优化目标。

5.6 复杂度与限制

设任务数为 V、依赖数为 E、离散时间跨度为 H：

- 确定性 Kahn：O((V + E) log V)；
- 关键路径 DP：O(V + E)；
- ready queue：O(V log V)；
- 每次区间查询和预留：O(log H)，寻找起点仍可能扫描多个时间点；
- 精确求解：最坏指数级，只适用于明确上限内的小实例。

教学版没有可抢占任务、多技能人员、日历、切换成本和动态事件。它重点展示“快速启发式 + 小规模精确 oracle”如何形成可解释的工程组合。

5.7 运行与阅读顺序

```powershell
dotnet run --project DataStructureAndAlgorithm.Scenarios.ProjectScheduling -c Release
dotnet run --project DataStructureAndAlgorithm.Scenarios.ProjectScheduling -c Release -- --
trace json
dotnet run --project DataStructureAndAlgorithm.Scenarios.ProjectScheduling -c Release -- --
trace mermaid
```

先在纸上完成拓扑序和关键剩余长度，再手算每次区间最大值。对小输入同时运行贪心和精确路径，尝试构造“关键路径优先并非最优”的反例。这比只阅读一组成功样例更能理解启发式边界。

6. 项目四：迷你存储引擎

6.1 应用场景与最小契约

迷你存储引擎模拟一个单进程、内存主导、可从日志恢复的键值库。它支持：

- Put(key, value) 插入或覆盖；
- TryGet(key, out entry) 点查；
- Delete(key) 逻辑删除；
- RangeScan(fromInclusive, toExclusive) 按键有序范围扫描；
- 重启后重放 WAL，恢复最后已提交状态。

这个项目把前面分散学习的树、概率结构、缓存和持久化协议放到同一条读写路径。重点不是复制成熟数据库，而是理解“索引、加速层、事实来源和恢复”为什么必须分工。

6.2 为什么主索引选择 B+ 树

B+ 树的内部节点只保存分隔键与子指针，实际记录集中在叶节点；叶节点按键有序并相互连接。因此：

- 点查沿树高下降，复杂度 O(log n)；
- B+ 树惰性范围扫描先定位起始叶，再按需沿叶链读取 k 个物理条目，约 O(log n + k)；消费者停止后枚举立即停止，不会按整棵树 Count 预分配或物化；
- 高扇出降低树高，也对应真实存储系统按页组织数据的思路。

分裂时要保持三个不变量：节点键有序；非根节点满足容量下界；父节点分隔键正确区分相邻子树。叶链还必须与树内中序顺序一致，否则点查可能正确而范围扫描漏项。测试应同时遍历树结构和叶链，不能只验证 Get。

教学版 B+ 树以内存对象模拟页，没有实现磁盘页编号、页缓存、合并回收和并发锁。它展示的是结构不变量，不是生产数据库页面管理器。

6.3 Bloom Filter 为什么只能回答“可能存在”

Bloom Filter 用多个哈希位置记录键。只要某一位为 0，键一定不存在；所有位都为 1，只能说明可能存在。读路径因此是：

1. Bloom 判断“一定不存在”时快速返回；
2. 判断“可能存在”时仍查询 B+ 树；
3. 绝不能把 Bloom 的阳性结果当作真实记录返回。

它允许假阳性，不允许假阴性。删除键时不能简单清零对应位，因为这些位可能被其他键共享；教学实现保持稳定过滤器语义，真实存在性始终由主索引决定。测试应断言所有已插入键从不被判为“不存在”，并把假阳性率作为统计观察，而不是要求永远为零。

6.4 LFU 缓存为什么不是事实来源

LFU 缓存优先淘汰访问频率低的记录，并用确定性规则处理相同频率。它只优化热点点查：缓存未命中必须回到 B+ 树；写入、覆盖和删除必须同步更新或失效缓存。

把缓存视为可丢弃派生状态很重要。崩溃恢复只重建主索引，不需要恢复 LFU 的历史频率；否则缓存会错误地参与持久化正确性。 教学实现的缓存值携带记录版本，并在写入时覆盖为最新命中、删除时覆盖为确定未命中；漏掉任一步都可能返回旧值，因此缓存一致性也是测试不变量。

6.5 为什么 WAL 必须先于内存修改

预写日志的顺序是：

```text
序列化操作 -> 追加 WAL -> 确认日志写入 -> 修改内存索引和缓存
```

如果先改内存再写日志，进程可能在两步之间崩溃：调用方看到写入成功的内存状态，却没有任何可恢复证据。WAL 先写让重启时可以顺序重放 Put 和 Delete，重建最终状态。

日志记录包含操作种类、键和值；Put 明确拒绝 null，Delete 使用独立操作与 tombstone，损坏的 Put-null 不会被悄悄解释成删除。序列号必须严格递增，重放同一已提交前缀得到相同最终状态。

每条 JSON 后的 LF 换行是最小提交标记。崩溃留下的最后一个无 LF 片段不会回放，并在重新追加前从最近提交边界截断；中间损坏或已经带 LF 的坏记录仍明确失败，不能静默跳过。 当前实现支持校验和与手动 checkpoint。新 WAL v2 用 Base64 保存 UTF-8 JSON 负载，并对原始负载计算 SHA-256；旧 v1 日志可继续读取，新写入使用 v2。它不承诺跨记录事务、中间记录修复、多写者并发或断电一致性。正式快照损坏会拒绝恢复，不能悄悄恢复为空库。

6.6 一次读写如何经过四种结构

写入路径：配置 WAL 时先追加并刷盘，再更新 B+ 树，随后把 LFU 覆盖为最新命中，并把键加入 Bloom。 删除路径同样先追加 Delete 记录，再在 B+ 树中写入 tombstone，并把 LFU 覆盖为确定未命中；Bloom 不承担删除真实性。

点查路径：先看 LFU；未命中时用 Bloom 排除确定不存在的键，此时直接返回且不回填缓存；仍可能存在时查 B+ 树。B+ 树命中时缓存值，遇到墓碑或 Bloom 假阳性时缓存确定未命中，避免热点不存在键反复访问树。 范围扫描直接使用 B+ 树叶链，因为 LFU 只覆盖热点个体，Bloom 也不支持顺序关系。

这个分工可以概括为：WAL 是恢复证据，B+ 树是有序事实索引，Bloom 是负查询加速器，LFU 是热点加速器。任何加速层都不能改变主索引语义。

6.7 复杂度、限制与运行

设记录数为 n、扫描返回数为 k、Bloom 哈希函数数为 h：

- B+ 树点查/写入：O(log n)；
- 存储范围扫描：O(log n + p)，p 是为得到最多 k 个活跃结果而经过的物理记录数；大量墓碑时 p 可显著大于 k；
- Bloom 查询/写入：O(h)；
- LFU 点查、写入和淘汰平均 O(1)；直接调用通用缓存的 Remove 时，极端情况下会扫描频率桶重算最小频率；
- 恢复：O((snapshotEntries + incrementalLogRecords) · indexUpdateCost)；检查点发布但旧日志尚未截断的中断恢复还需扫描旧日志。

```powershell
dotnet run --project DataStructureAndAlgorithm.Scenarios.MiniStorage -c Release
dotnet run --project DataStructureAndAlgorithm.Scenarios.MiniStorage -c Release -- --trace
json
dotnet run --project DataStructureAndAlgorithm.Scenarios.MiniStorage -c Release -- --trace
mermaid
```

建议先执行“写入 A、写入 B、覆盖 A、删除 B”，手算 WAL 和最终键空间，再创建新引擎重放日志。随后检查点查、范围扫描和缓存命中是否与恢复前一致。

6.8 手动检查点与日志压缩

调用 engine.Checkpoint() 会返回版本、存活条目数、回收墓碑数和日志压缩前后字节数。纯内存模式不能创建检查点。

1. 按键序构建只包含存活条目的快照和新的 B+ 树，保留每个键原有版本。
2. 向 WAL 同目录的临时文件写入校验快照，完成刷盘后替换正式快照。
3. 正式快照提交后才截断旧 WAL，重建 Bloom Filter 并清空 LFU。
4. 恢复时先读取快照，再应用版本大于快照版本的增量日志；已包含的旧日志不会重复应用。

StorageStatistics.SnapshotVersion、SnapshotEntriesLoaded 和 WalRecordsReplayed 分别表示快照版本、加载条目数与实际重放的增量记录数。全局版本不因压缩而重置，运行期间的业务计数也不清零。

如果快照刚发布但日志还没截断，旧日志仍然有效，恢复时由版本边界跳过。末尾半写记录可以截断，已提交的损坏记录、未知格式和序列缺失则明确失败。引擎在读取或修复持久化文件之前取得独占写锁；第二个写实例无法打开同一路径。持久化写入失败后必须关闭并重开引擎。

MiniStorageCheckpointTests 使用独立的版本化字典模型，交错执行写入、删除、读取、检查点和重启。测试专用子进程在日志半写、日志刷盘、快照半写、快照刷盘、快照发布、日志截断及检查点完成处等待，父进程强制终止后验证恢复。I/O 异常通过内部接口注入，不改变公开 API。

StorageScalingScenarioBenchmarks 在 100 / 1,000 个存活键与每键 1 / 10 次历史修改下比较完整日志和快照增量恢复；文件准备不在计时区间内。快照保留的是当前键空间，因此恢复成本由存活键数和增量操作数决定，不再一直随完整历史增长。

7. 四个项目共同展示的 C# 最佳实践

7.1 用不可变记录表达快照结果

输入和输出优先使用 record / record struct，让值相等语义和解构更自然。内部索引仍可变，但只通过引擎方法维护不变量。返回集合采用只读视图或快照，防止调用方绕过 API 修改历史结果。

7.2 参数验证放在公共边界

公共方法使用 ArgumentNullException.ThrowIfNull、ArgumentOutOfRangeException 和带参数名的 ArgumentException。验证一次后，核心循环依赖已建立的前置条件，避免到处重复防御。精确调度的规模限制、扫描区间顺序和空键策略都属于公开契约。

7.3 现代语法服务于意图，而不是炫技

C# 15 集合表达式、模式匹配、目标类型 new、record 和 required 成员可以减少样板代码，但算法的边界和状态转移仍应显式。复杂循环宁可使用有含义的局部变量，也不要把 BM25、残量边更新或线段树下推压成难以断点调试的一行表达式。

7.4 注释解释“为什么”

高价值注释应解释：为什么区间是左闭右开、为什么 Bloom 阳性仍要查树、为什么反向残量边费用取负、为什么精确求解有规模上限。低价值注释只是把 index++ 翻译成“索引加一”。教学代码的目标是让读者能从注释恢复推理，而不是让每行都有中文。

7.5 不依赖哈希枚举顺序

所有用户可见结果、追踪状态键和相同分数候选都要定义显式比较器。比较器逐字段调用 CompareTo，不使用整数相减，避免溢出。确定性不是算法正确性的全部，却是跨平台测试、差分比较和性能趋势可解释的前提。

7.6 缓存与概率结构不能拥有业务真相

路线 LRU、查询缓存、LFU 和 Bloom 都是可重建加速层。缓存键必须覆盖影响结果的版本状态，写入和删除必须维护失效规则；任何缓存未命中或 Bloom 阳性都不能改变领域结果。先实现无缓存正确路径，再加入加速层并做等价测试。

7.7 可选观察器不能反向破坏领域提交

四个场景把注入的 IAlgorithmTraceSink 包装为 BestEffortAlgorithmTraceSink。普通日志、UI 或测试接收器异常只增加丢弃计数，不会让“索引或 WAL 已提交”的操作向调用方报失败；否则调用方重试可能制造重复写入或搜索双状态。内存耗尽、栈溢出和访问冲突等进程级故障仍然传播，不能伪装成普通日志丢失。

回归测试使用必定抛异常的 sink，逐个运行配送、搜索、调度和带 WAL 的存储，并在存储重启后再次核对已提交值。这个测试证明的是隔离边界，不是说追踪丢失无需监控；显式创建装饰器的调用方仍可检查 DroppedEventCount。

8. 把正确性升级为“持续可证明”

这里的“证明”不是声称测试等价于形式化证明，而是建立一条持续运行、彼此独立的证据链。每次代码变化都重新检查契约、不变量、参考模型、可观察性和性能趋势，使错误更早、更容易定位。

8.1 第一层：把不变量写成可执行断言

每个结构都先列出不因输入变化而改变的规则：B+ 树键序与叶链一致、线段树父节点最大值正确、残量容量非负、匹配端点唯一、位置列表递增、WAL 重放顺序稳定。测试不应只断言最终 JSON，而应在最小操作后检查这些结构性规则。

不变量测试回答“结果是否自洽”，但不一定回答“是否为最优答案”。因此还需要独立 oracle。

8.2 第二层：模型、性质与差分测试

- 模型测试：用简单但明显正确的结构作参照，例如 SortedDictionary 对照 B+ 树、直接数组对照懒标记树。
- 性质测试：让 FsCheck 生成一族小输入，检查排序性、幂等性、恢复前后等价、容量不超限等普遍规则。
- 差分测试：让优化实现与不同思路的 oracle 处理同一输入，例如最小费用流对照小规模匹配穷举、BM25 候选对照直接扫描、精确调度对照更小规模的开始时间完整枚举。
- 状态机测试：生成 Put/Get/Delete/Scan 或缓存访问序列，每一步都与参考模型比较，而不是只比较最终状态。

安全阀也要可测试：精确调度的生产预算保持两百万节点，测试通过 internal 注入一个极小预算，确定性触发 ExactSearchAborted 与异常；这样验证的是语义，而不是 CI 机器能否快速跑完两百万节点。

为什么强调“小而可缩减”？随机大输入失败后很难理解。受控小输入既能覆盖大量组合，又允许 FsCheck 缩减为最小反例。oracle 必须比被测实现更简单且思路不同；复制同一算法只会让两个实现共享同一错误。

8.3 第三层：稳定的 trace schema

四个场景共用可选 IAlgorithmTraceSink。JSON 模式顶层保持 { result, trace }，事件包含显式 schema version、步骤、算法、阶段、说明和只读状态。schema version 的意义是：当事件字段或语义未来变化时，消费者可以明确拒绝、迁移或兼容，而不是静默误读。

端到端 CLI 契约直接解析整段 stdout，字段名区分大小写且禁止静默扩展；事件 step 必须严格等于 1..N。场景内部再用 best-effort 装饰器隔离接收器异常，使“可观察”既稳定又不会改变领域提交。

追踪契约应验证：

1. 开启追踪前后领域结果完全相同；
2. 步骤从 1 连续递增；
3. 状态快照不会被后续修改回写；
4. 数值使用 InvariantCulture，状态键按 Ordinal 排序；
5. Mermaid 正确转义引号、反斜线和换行，并保持稳定末尾换行；
6. 非法命令行模式以约定退出码失败。

追踪只记录阶段和领域决策，不记录每次比较或树节点访问。否则观察本身会改变复杂度并淹没有价值的信息。trace 证明的是“观察契约没有漂移”，不能替代算法结果测试。

8.4 第四层：跨平台 CI

同一 Release 构建和测试应至少在 Windows 与 Linux 运行。跨平台矩阵能发现路径分隔符、换行、大小写、区域性数字格式和文件锁等本地单机难以暴露的问题。质量门禁还应包括格式、分析器、覆盖率和关键模块变异测试，但不要把覆盖率百分比误解为正确率。

CI 失败时先区分：编译/测试是真实代码错误，还是文档换行、coverage 路径和 shell glob 等平台差异。修复基础设施后仍必须保留相同语义门禁，不能通过跳过测试获得绿色状态。

当前验证快照（2026-09-12）为 Windows/Linux 508/508 测试通过，行覆盖率 95.38%、分支覆盖率 86.86%，覆盖率显式包含四个综合场景。前端使用状态测试、类型检查和三个浏览器引擎的交互测试。变异测试按 core、delivery、search、scheduling、storage 分组运行；稳定契约是全部 Release 测试通过、行覆盖率至少 80%、分支覆盖率至少 70%，每组 Basic 变异分数不低于 60% break 阈值。具体测试数和分数以最新报告为准。

8.5 第五层：观察基准趋势，不设脆弱时间阈值

保留 ScenarioBenchmarks 的六条小样本烟雾工作负载，新增 Search、Delivery、Scheduling、Exact、Storage 参数化分组。搜索使用 100 / 1,000 / 10,000 篇文档，区分建索引、兼容查询冷缓存、命中与 BM25 排名；配送比较 20 / 100 地点和出度 2 / 8 下的首次规划与路线缓存；调度比较 20 / 100 任务的贪心及 4 / 6 / 8 任务的精确解；存储比较 100 / 1,000 个键、每键 1 / 10 次历史修改的检查点与恢复，并独立测量冷热读取。

PR 执行 Dry，周期任务执行 Short，手动任务可选完整采样。报告保留 90 天，tools/benchmark_report.mjs 要求方法、参数、作业、SDK、运行时、操作系统及处理器一致才生成历史变化百分比。没有匹配历史时明确标为无可比基线；Dry 只证明工作负载可运行。

不要在共享 CI 中写“必须小于 2 ms”这类硬阈值。虚拟机负载、CPU 频率和运行时升级都会产生噪声。更可靠的方式是上传结构化结果、固定数据规模，按周观察中位数和分配量趋势；只有跨多次运行的显著回退才进入调查。

8.6 证据链如何配合

证据	擅长发现	不能单独证明
示例/回归测试	已知边界和历史缺陷	未见过的输入族
不变量测试	结构是否自洽、约束是否违反	结果是否全局最优
模型/差分测试	优化实现与独立答案是否一致	超出 oracle 规模的全部输入
性质/状态机测试	操作序列与输入族问题	生产环境所有故障
变异测试	断言是否能杀死有意义的错误	没有建模的需求
trace contract	可观察性与输出协议是否稳定	算法本身正确性
基准趋势	时间和分配回退	语义正确性

持续可证明的关键不是选择一种“最强测试”，而是让不同证据尽量独立。每增加一个优化，都要保留简单正确路径或小规模 oracle，验证优化没有改变语义。

9. 推荐的八周学习路线

第一周：契约、不变量与追踪

运行四个项目的普通 JSON、追踪 JSON 和 Mermaid。为每个项目写出输入契约、领域目标、三个核心不变量和一个模型限制。确认开启追踪不改变领域结果。

第二周：配送 V2

手算 A* 候选、最大匹配和同流量最小费用方案。自己实现一个两车两单穷举 oracle，再解释残量反向边为什么允许纠正早期选择。

第三周：搜索 V2

手工建立位置倒排表，实现两个有序位置列表的短语/邻近合并。分别比较词频排序和 BM25，画出 BK-tree 查询时保留的距离区间。

第四周：调度 V2

先写 DAG 和资源可行性检查，再实现区间加/最大值懒标记。构造一个贪心非最优反例，用小规模精确解量化差距，并解释规模上限。

第五周：迷你存储引擎

依次学习 B+ 树叶链、Bloom 单向保证、LFU 失效和 WAL 顺序。对同一操作序列比较运行中状态、重启恢复状态和 SortedDictionary 参考模型。

第六周：性质与状态机

为四项目各设计至少一条性质。生成器保持输入小且合法，oracle 使用直接扫描、完整排序、简单数组或穷举，不复制被测优化算法。观察失败时如何缩减。

第七周：跨平台契约与性能趋势

阅读 CI 矩阵，理解 Windows/Linux 差异。运行短基准只验证基准可执行，再进行正式多轮采样；记录输入规模、运行时和分配量，不凭单次数字下结论。

第八周：隐藏实现重写

只保留公开 API、测试和本指导，独立重写一个项目的核心路径。完成后回答：目标函数是什么、不变量是什么、复杂度瓶颈在哪里、哪个优化由什么证据支持、生产化还缺什么。

10. 可继续扩展的方向

配送

- 在当前 Johnson 势能逐次最短增广路之外，比较 cost-scaling、容量缩放和整数费用表示的适用条件与工程代价。
- 加入时窗、车辆多单路线和容量累积，比较 VRP 与当前二分匹配模型。
- 设计地图增量版本与缓存精细失效。

搜索

- 支持文档删除、段合并和压缩 posting。
- 比较 BM25 参数与字段权重，加入标题/正文分域。
- 在现有 Rune 边界之上加入 Unicode normalization、语言相关 tokenizer 和同义词扩展。

调度

- 加入可抢占、多资源种类和任务切换成本。
- 比较分支限界、动态规划、整数规划和 CP-SAT。
- 为下界质量和搜索节点数建立基准趋势。

存储

- 在已实现校验和与 checkpoint 的基础上，增加页编号、页缓存和自动维护策略。
- 处理截断日志与部分写入，区分 logical recovery 和 durability。
- 研究 MVCC、快照隔离、LSM Tree 与 B+ Tree 的读写权衡。

11. 阶段验收清单

完成本阶段后，应能在不看实现的情况下解释并完成：

- 把“最大数量，其次最小成本”翻译为最大流量上的最小费用目标。
- 说明弱连通预筛为何不能替代有向最短路。
- 从位置倒排表推导短语和邻近查询，并解释 BM25 的三个权重来源。
- 说明 BK-tree 的剪枝依据及其最坏情况。
- 写出懒标记区间加的最大值、最小值与 Int128 延迟增量不变量，并使用左闭右开区间避免边界歧义。
- 区分调度可行解、启发式解和小规模精确最优解。
- 解释 B+ 树惰性叶链、Bloom 假阳性、LFU 饱和频率与 WAL 先写/换行提交顺序。
- 为每个综合项目设计不同于被测实现的简单 oracle。
- 说明示例、性质、差分、变异、trace contract 和基准趋势分别能证明什么。
- 在 Windows 与 Linux 上保持确定性输出和稳定追踪协议。
- 明确指出四个教学项目距离生产系统仍缺少的并发、持久化、规模和运维能力。

真正掌握的标志不是记住四条流水线，而是需求变化后仍能从目标函数、契约、不变量和复杂度重新推导设计，并能用持续运行的证据说明“为什么相信它仍然正确”。